

Product Backlog - VitrineCol

Compilação Técnica Editorial

Sumário

Capítulo 01 - Product Backlog: VitrineCol(plataforma web de moda local)

Capítulo 01: Product Backlog:

VitrineCol(plataforma web de moda local)

Origem: product-backlog.md

Product Backlog: VitrineCol(plataforma web de moda local)

Este é o seu quadro de trabalho: **54 cartões, um por aula**, em **18 dias de aula**. Cada cartão repete, em forma de tarefa, o passo a passo da aula correspondente em [docs/guia-aulas/](#). Abra a aula, siga o cartão e só mova para "Concluído" quando todos os critérios de aceite passarem.

Escopo: Dias 1 a 15 (Aulas 1 a 45) e Dias 26 a 28 (Aulas 76 a 84). Os Dias 16 a 25 e 29 a 31 tratam de outros assuntos e **não** têm cartões aqui (por isso a numeração das aulas pula de 45 para 76).

Como ler os códigos: **RF** é um requisito funcional (o que o sistema faz), **RN** é uma regra de negócio (uma regra que o sistema sempre respeita) e **CT** é um caso de teste. A descrição de cada um está no anexo "Requisitos do sistema" da ementa e, no fim deste arquivo, há a tabela de cobertura dos RF.

Como ler o nome de um cartão: **D5-A13** quer dizer **Dia 5, Aula 13**.

Parte 1: Como configurar o Trello

As listas do quadro

Crie cinco listas, da esquerda para a direita:

Lista	Quando o cartão fica aqui
Backlog	Cartões dos próximos dias, ainda sem data de hoje.
A Fazer Hoje	Os três cartões do dia de aula (um por aula) e os cartões atrasados que ficaram de ontem.
Em Andamento	A dupla começou o cartão e está seguindo a aula.
Em Revisão	A dupla conferiu todos os critérios de aceite do cartão e o commit foi enviado (<code>git push</code>). Se o cartão não tem código (por exemplo, wireframes ou Figma), o material foi guardado onde a equipe acessa.
Concluído	Outra integrante da equipe olhou o resultado, concordou que os critérios passam e arrastou o cartão para cá.

Regra de mudança de lista (resumo): Backlog → A Fazer Hoje (de manhã, no começo do dia) → Em Andamento (ao começar a aula) → Em Revisão (critérios conferidos e commit enviado) → Concluído (outra integrante confirmou).

Limite de trabalho em andamento

- **No máximo 1 cartão por dupla** na lista "Em Andamento". Terminou? Mova para "Em Revisão" antes de puxar o próximo.
- Os cartões dependem uns dos outros: o campo "Depende de" diz qual cartão precisa estar em "Concluído" antes de você começar.

Cartão atrasado

Se o cartão **não ficou pronto no dia**:

1. Ele vai para o **início** da lista "A Fazer Hoje" no dia seguinte (antes dos cartões novos).
2. A dupla pede ajuda ao professor ou usa a seção **"Se travar"** da aula.
3. **Nunca pule para o próximo cartão sem concluir o anterior**: cada aula parte do que a anterior deixou pronto.
4. O trabalho que não coube nos 60 minutos fica registrado em **"Observações do dia"**; ele não vira cartão extra.

Os passos de **segurança** (chaves, RLS, política de fotos) e de **banco** (scripts SQL) nunca são pulados, mesmo com atraso.

Etiquetas (Labels)

Crie estas dez etiquetas de **assunto**. Use uma ou duas por cartão.

Etiqueta	Para que serve
Configuração	Instalar ferramentas, criar repositório, projeto e ajustes do ambiente.
Front-end (HTML/CSS)	Páginas, estilos, layout, responsividade e telas no Figma.
JavaScript	Scripts das páginas, DOM, eventos e validações.
Orientação a Objetos	Classes, herança, polimorfismo, agregação e ErroApp .
Supabase (Banco de Dados)	Tabelas, SQL, consultas, Storage e ligação do site ao Supabase.
Segurança	Login, RLS, chaves, proteção de telas e testes de acesso indevido.
Fluxo de Pedido	Sacola, finalização, pedidos por loja, status e avisos.
Testes	Casos de teste, depuração e correções.
Publicação	GitHub Pages, tags de versão e endereço publicado.
Documentação	Backlog, README, guia de estilo e relatórios.

Observação: o projeto **não tem servidor próprio** (o "back-end" é o Supabase), por isso **não existe** a etiqueta "Backend".

Etiquetas de prioridade

Crie mais três etiquetas e coloque **uma** em cada cartão:

Prioridade	Quer dizer
Essencial	O sistema não funciona sem isso. Não pode sair.
Importante	Faz muita falta, mas o sistema ainda funciona sem isso.
Desejável	Melhora o sistema; é o primeiro a sair se o tempo apertar.

Se o tempo apertar, o que sai primeiro (nesta ordem): (1) o link do mapa na página da loja; (2) o carrossel da página inicial, o aviso da lojista à cliente por WhatsApp e a reordenação das fotos do produto; (3) o filtro por tamanho; (4) o envio de fotos pelo sistema (usando o endereço de uma imagem já hospedada).

Prazo (Due date)

Cada equipe **copia o quadro** e usa a **data do cartão como prazo** (Due date). O dia de aula termina com os três cartões do dia em "Concluído" ou, se não deu, atrasados com a regra acima.

Parte 2: O Backlog Distribuído por Dias

Tabela-resumo

Dia	Data	Aulas	Cartões	Entrega do dia / Marco
1	06/10/2026 (terça-feira)	1 a 3	D1·A1, D1·A2, D1·A3	Backlog, repositório da equipe e primeira página
2	07/10/2026 (quarta-feira)	4 a 6	D2·A4, D2·A5, D2·A6	Protótipo e guia de estilo · Marco 1
3	08/10/2026 (quinta-feira)	7 a 9	D3·A7, D3·A8, D3·A9	Páginas do projeto em HTML, sem estilo
4	09/10/2026 (sexta-feira)	10 a 12	D4·A10, D4·A11, D4·A12	Páginas estilizadas conforme o guia de estilo
5	13/10/2026 (terça-feira)	13 a 15	D5·A13, D5·A14, D5·A15	Front-end estático responsivo (versão 0.1) · Marco 2
6	14/10/2026 (quarta-feira)	16 a 18	D6·A16, D6·A17, D6·A18	Interações básicas nas páginas
7	15/10/2026 (quinta-feira)	19 a 21	D7·A19, D7·A20, D7·A21	Catálogo filtrável com dados fictícios
8	16/10/2026 (sexta-feira)	22 a 24	D8·A22, D8·A23, D8·A24	Classes de domínio do projeto
9	19/10/2026 (segunda-feira)	25 a 27	D9·A25, D9·A26, D9·A27	Banco criado e populado
10	20/10/2026 (terça-feira)	28 a 30	D10·A28, D10·A29, D10·A30	Catálogo lendo do banco
11	21/10/2026 (quarta-feira)	31 a 33	D11·A31, D11·A32, D11·A33	CRUD com imagens (versão 0.5) · Marco 3
12	22/10/2026 (quinta-feira)	34 a 36	D12·A34, D12·A35, D12·A36	Páginas de produto e loja, sacola por loja e pedidos por loja com WhatsApp

Dia	Data	Aulas	Cartões	Entrega do dia / Marco
13	23/10/2026 (sexta-feira)	37 a 39	D13-A37, D13-A38, D13-A39	Login, recuperação de senha e segurança básica; pedido gravado
14	26/10/2026 (segunda-feira)	40 a 42	D14-A40, D14-A41, D14-A42	Página inicial de vitrine e fluxo completo testado
15	27/10/2026 (terça-feira)	43 a 45	D15-A43, D15-A44, D15-A45	MVP publicado (versão 0.9) e UC3 avaliada · Marco 4
26	12/11/2026 (quinta-feira)	76 a 78	D26-A76, D26-A77, D26-A78	Sistema com conteúdo real e acompanhamento do pedido
27	13/11/2026 (sexta-feira)	79 a 81	D27-A79, D27-A80, D27-A81	Relatório de testes
28	16/11/2026 (segunda-feira)	82 a 84	D28-A82, D28-A83, D28-A84	Versão 1.0 publicada e congelada · Marco 8

Totais: 18 dias, 54 aulas, 54 cartões (3 por dia). Não há aula em 12/10/2026 (feriado) nem em 17 e 18 de outubro (fim de semana).

Marcos: Marco 1 (D2-A6), Marco 2 (D5-A15), Marco 3 (D11-A33), Marco 4 (D15-A45) e Marco 8 (D28-A84). Os demais marcos do curso (5 a 7 e 9) acontecem nos dias que não fazem parte deste backlog.

Dia 1 · 06/10/2026 · terça-feira

Entrega do dia: backlog do MVP, repositório da equipe e primeira página aberta no Live Server.

Observações do dia: - A Aula 2 (Git e GitHub) é a mais longa do dia: instalar ferramentas, criar o repositório e fazer o primeiro commit de **cada** integrante. Se a equipe não terminar, o que sobrar (clones e commits das últimas integrantes) é feito em casa antes da Aula 3. - O guia da Aula 2 monta o quadro Kanban no GitHub Projects. Aqui o quadro principal é o **Trello** (Parte 1). Quem quiser pode criar também o quadro do GitHub, mas não é obrigatório para fechar o cartão.

Data: 06/10/2026 - terça-feira - **Cartão (Título no Trello):** D1·A1 – Montar o quadro do Trello, priorizar os 25 requisitos e formar a equipe - **User Story:** Como aluna desenvolvedora, Quero ter o quadro da equipe pronto e a lista dos 25 requisitos (RF-01 a RF-25) priorizada, Para que todas saibam o que vamos construir e o que sai primeiro se faltar tempo. - **Critérios de Aceite:** - O quadro do Trello da equipe existe, com as 5 listas (Backlog, A Fazer Hoje, Em Andamento, Em Revisão, Concluído) e as etiquetas da Parte 1. - Existe um documento (Google Docs, Planilhas, Word, Excel ou Bloco de Notas) com a tabela dos 25 requisitos e a prioridade de cada um. - A equipe diz em voz alta os requisitos Essenciais (RF-01, 02, 03, 06, 08, 09, 10, 12, 13, 14, 15, 16, 17, 18 e 21). - Os cinco papéis (Produto/negócio, UX/UI, Front-end, Banco/back-end, Conteúdo/QA) têm uma responsável nesta semana, e a equipe sabe quando o papel muda. - A equipe responde sem olhar a tabela: "Se faltar tempo, o que sai primeiro?" (1º link do mapa; 2º carrossel e reordenação das fotos; 3º filtro por tamanho; 4º envio de fotos pelo sistema). - **Checklist interna do cartão:** - [] Copiar o quadro modelo do Trello para a conta da equipe e conferir as 5 listas e as etiquetas (Parte 1). - [] Abrir o editor de texto ou a planilha onde o backlog vai ficar (Google Docs, Google Planilhas, Word, Excel ou Bloco de Notas). - [] Copiar a tabela dos 25 requisitos (código, o que o sistema deve fazer, prioridade) para o documento. - [] Dividir os 25 requisitos entre as integrantes (por exemplo, 5 para cada uma); cada uma confere a sua parte e depois todas revisam juntas. - [] Para cada requisito, perguntar "o sistema funciona sem isso?". Se a resposta for sim, ele não é Essencial. - [] Marcar na tabela os requisitos Essenciais, Importantes e Desejáveis, sem deixar todos como Essenciais. - [] Combinar a regra "se faltar tempo, o que sai primeiro" e escrevê-la no documento. - [] Formar a equipe (de 4 a 5 pessoas) e anotar em uma tabela quem assume cada um dos cinco papéis nesta semana e quando o papel muda (sugestão: troca a cada dia de aula). - [] Combinar que as duplas se alternam (uma digita, a outra confere) e que todas escrevem código. - [] Escrever os combinados da equipe: grupo de mensagens, horário de encontro fora da aula e a regra "ninguém fica com dúvida sozinha por mais de 15 minutos". - **Etiqueta sugerida:** Documentação, Configuração · Essencial - **Depende de:** nenhum - **Aula e requisitos:** docs/guia-aulas/dia01-aula01-backlog-do-mvp-e-formacao-das-equipes.md · visão geral de RF-01 a RF-25 (só leitura, sem implementação)

Data: 06/10/2026 - terça-feira - **Cartão (Título no Trello):** D1·A2 – Instalar Git e Live Server, criar o repositório vitrine-col e fazer o primeiro commit de cada integrante - **User Story:** Como aluna

desenvolvedora, Quero um repositório compartilhado no GitHub com o meu primeiro commit, Para que a equipe guarde o histórico do código e trabalhe junta sem perder versões. - **Critérios de Aceite:** - No terminal do VS Code, `git --version` mostra um número de versão. - O botão **Go Live** (em português: **Ir ao vivo**) aparece no canto inferior direito do VS Code. - A página do repositório `vitrine-col` no GitHub mostra `README.md` e `.gitignore`. - O README lista o nome de cada integrante da equipe. - `git log --oneline` mostra pelo menos um commit seu, com mensagem em português. - **Checklist interna do cartão:** - [] Abrir o terminal do VS Code: **Terminal > New Terminal** (em português: **Terminal > Novo Terminal**) e rodar `git --version`. Se der "comando não encontrado", instalar o Git (Windows: <https://git-scm.com/download/win>, com as opções padrão) e reabrir o VS Code. - [] Instalar o Live Server: abrir **Extensions** (em português: **Extensões**, atalho Ctrl+Shift+X; no Mac, Cmd+Shift+X), buscar `Live Server`, escolher a de **Ritwick Dey** e clicar em **Install** (em português: **Instalar**). - [] Rodar `git config --global user.name "Seu Nome"` e `git config --global user.email "seu-email@exemplo.com"` (o mesmo e-mail da conta do GitHub). - [] (Uma integrante por equipe) No GitHub, clicar em + > **New repository** (em português: **Novo repositório**), nomear `vitrine-col`, deixar **Public** (em português: **Público**) e **não** marcar **Add a README file**; clicar em **Create repository** (em português: **Criar repositório**). - [] Convidar as colegas em **Settings > Collaborators > Add people** (em português: **Configurações > Colaboradores > Adicionar pessoas**) e conferir que todas aceitaram o convite por e-mail. - [] Criar a pasta `vitrine-col` no computador e abrir no VS Code: **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**); se perguntar, clicar em **Yes, I trust the authors** (em português: **Sim, eu confio nos autores**). - [] No painel **Explorer** (em português: **Explorador**), criar `README.md` e `.gitignore` com o conteúdo da aula e salvar com Ctrl+S (Mac: Cmd+S). - [] Rodar `git init` e os comandos da aula para o primeiro commit (mensagem em português) e o `git push`; entrar no GitHub se o navegador pedir. - [] Atualizar a página do repositório no GitHub e conferir que os dois arquivos aparecem. - [] (Cada outra integrante, uma de cada vez) Rodar `git clone https://github.com/SEU-USUARIO/vitrine-col.git` e abrir a pasta com **File > Open Folder...** (em português: **Arquivo > Abrir Pasta...**). - [] Editar o `README.md` trocando `- Nome da primeira integrante` pelo seu nome (uma linha por integrante), salvar e rodar `git add .`, `git commit`, `git pull` e `git push`. - [] Se aparecer conflito (`<<<<<<`, `=====`, `>>>>>>`), apagar as três marcas, manter as linhas de todas, rodar `git add .`, `git commit -m "Resolve conflito no README"` e `git push`. - [] Montar o quadro Kanban como a aula pede (GitHub: aba **Projects > New project > Board**, em português: **Projetos > Novo projeto > Quadro**) ou conferir que o quadro do Trello (D1-A1) já tem um cartão por requisito Essencial. - [] Rodar `git log --oneline` e conferir um commit de cada integrante. - **Etiqueta sugerida:** Configuração · Essencial - **Depende de:** D1-A1 - **Aula e requisitos:** docs/guia-aulas/dia01-aula02-git-e-github-repositorio-commits-e-quadro-kanban.md · não se aplica (preparação do ambiente de desenvolvimento)

Data: 06/10/2026 - terça-feira - **Cartão (Título no Trello):** D1·A3 – Criar a primeira página HTML e abrir com o Live Server - **User Story:** Como aluna desenvolvedora, Quero escrever a primeira página em HTML e vê-la no navegador, Para que eu entenda a estrutura básica de uma página e o ciclo salvar-e-ver do Live Server. - **Critérios de Aceite:** - O `index.html` abre pelo Live Server (endereço parecido com `http://127.0.0.1:5500/index.html`) com título, dois parágrafos, a imagem e o link. - A aba do navegador mostra o texto "VitrineCol". - Ao trocar um texto, salvar (Ctrl+S) e olhar o navegador, a mudança aparece sem apertar nada. - Se o `src` da imagem estiver errado, aparece o texto do `alt`. - `git log --online` mostra o commit da aula e o GitHub mostra `index.html` e a pasta `imagens`. - **Checklist interna do cartão:** - [] Abrir a pasta `vitrine-col` no VS Code e rodar `git status`: deve dizer `nothing to commit, working tree clean`. - [] No **Explorer** (em português: **Explorador**), clicar em **New Folder** (em português: **Nova Pasta**) e criar a pasta `imagens`. - [] Criar `imagens/sem-foto.svg` com o conteúdo da aula. - [] Criar `index.html` na raiz com o conteúdo da aula e salvar (Ctrl+S; Mac: Cmd+S). - [] Clicar em **Go Live** (em português: **Ir ao vivo**) ou clicar com o botão direito no arquivo e escolher **Open with Live Server** (em português: **Abrir com Live Server**). - [] Testar a atualização automática: trocar o texto de um parágrafo, salvar e olhar o navegador. - [] Experimento 1: trocar `<h1>` por `<h2>` (e o fechamento), ver o título menor e voltar para `h1`. - [] Experimento 2: apagar o valor do `alt` da imagem, entender por que leitores de tela precisam dele e colocar o texto de volta. - [] Experimento 3: trocar o `src` por `imagens/nao-existe.svg`, ver o `alt` aparecer e voltar para `imagens/sem-foto.svg`. - [] Se algo falhar, apertar F12 no navegador e ler a aba **Console** (em português: **Console**). - [] Rodar `git add .`, `git commit -m "..."` (mensagem em português) e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D1·A2 - **Aula e requisitos:** docs/guia-aulas/dia01-aula03-html-estrutura-da-pagina-textos-links-e-imagens.md · não se aplica (base de todas as telas)

Dia 2 · 07/10/2026 · quarta-feira

Marco: Marco 1 · protótipo navegável e guia de estilo prontos (junto com o backlog do MVP e o quadro do dia 1). Evidência: arquivo do Figma e quadro Kanban.

Entrega do dia: protótipo navegável no Figma e guia de estilo.

Observações do dia: - Nenhum arquivo de código novo no repositório hoje: o trabalho vive no papel e no Figma. A única mudança no repositório é o `README.md` com os links do protótipo e do guia de estilo (Aula 6). - O Figma e o verificador de contraste têm menus em inglês; os nomes aparecem como na tela, com a versão em português ao lado.

Data: 07/10/2026 - quarta-feira - **Cartão (Título no Trello):** D2-A4 – Desenhar os wireframes do Catálogo, da Loja, do Produto e do Login - **User Story:** Como aluna desenvolvedora, Quero desenhar em baixa fidelidade as quatro telas-chave e o caminho do catálogo até o pedido, Para que a equipe combine como as telas serão antes de escrever mais código. - **Critérios de Aceite:** - Existe um wireframe de cada uma das 4 telas (catálogo, loja, produto, login), com todos os campos e botões escritos. - Há um desenho do caminho com setas, do catálogo até o WhatsApp, com no máximo 6 ações da cliente. - Outra equipe respondeu às 5 perguntas de revisão e a equipe anotou os ajustes. - As fotos dos wireframes estão salvas em uma pasta que toda a equipe acessa. - **Checklist interna do cartão:** - [] Pegar 4 folhas (ou 4 áreas de desenho) e escrever no topo: Catálogo, Loja, Produto, Login. Desenhar em formato de celular (retângulo alto, cerca de 9 cm por 16 cm). - [] Combinar as convenções: retângulo com "X" = foto; linhas onduladas = texto; retângulo com texto = botão; retângulo vazio = campo de digitar. - [] Desenhar o Catálogo (RF-01 a RF-04): cabeçalho, filtros por tipo, loja e tamanho, cards de produto e botão "Carregar mais produtos". - [] Desenhar a Loja: nome, descrição, endereço, botões "Conversar no WhatsApp" e "Ver no mapa" e os cards dos produtos da loja. - [] Desenhar o Produto: foto grande, faixa de miniaturas, preço, "Vendido por [loja]", descrição, botões de tamanho (um deles "sem estoque"), campo de quantidade e botão "Adicionar à sacola". - [] Desenhar o Login: título "Entrar", campos E-mail e Senha, botão "Entrar" e links "Esqueci minha senha" e "Ainda não tem conta? Cadastre-se". - [] Usar setas para indicar "esta imagem leva a esta tela" (o card leva ao Produto; o nome da loja leva à Loja). - [] Em uma folha à parte, desenhar o caminho Catálogo → Produto → Sacola → (se não estiver logada) Login → Pedidos enviados → WhatsApp. - [] Contar as ações da cliente (clicar no produto, escolher tamanho, Adicionar à sacola, abrir a Sacola, Finalizar sacola, botão do WhatsApp) e simplificar se passar de 6. - [] Trocar os rascunhos com outra equipe e pedir que responda, sem explicação: qual é o botão principal de cada tela? Dá para saber de qual loja é cada produto? Dá para voltar ao catálogo de qualquer tela? O que aparece se a lista estiver vazia? O caminho passa de 6 ações? - [] Ajustar os desenhos com as respostas, tirar uma foto de cada wireframe final com o celular e guardar na pasta compartilhada. - **Etiqueta sugerida:** Front-end (HTML/CSS), Documentação · Importante - **Depende de:** D1-A3 - **Aula e requisitos:** docs/guia-aulas/dia02-aula04-wireframes-das-telas-chave.md · não se aplica (planejamento das telas; prepara RF-01, RF-06, RF-08 e RF-13)

Data: 07/10/2026 - quarta-feira - **Cartão (Título no Trello):** D2-A5 – Montar o protótipo navegável no Figma (Catálogo, Produto e Sacola) - **User Story:** Como aluna desenvolvedora, Quero um protótipo clicável das telas no Figma, Para que a equipe e outras pessoas testem o caminho catálogo, produto e sacola antes da programação. - **Critérios de Aceite:** - O arquivo do Figma tem 3 quadros nomeados (Catálogo, Produto, Sacola) e pelo menos 2 componentes (cabeçalho e card de produto). - No modo **Present** (em português: **Apresentar**), os cliques levam Catálogo →

Produto → Sacola → Catálogo. - Uma colega concluiu a tarefa "escolha uma peça e finalize a sacola" sem ajuda, e a equipe anotou pelo menos um ajuste. - O link de visualização do protótipo está no [README.md](#) ou no quadro, e abre sem pedir login. - **Checklist interna do cartão:** - [] Entrar no Figma (<https://www.figma.com>) e criar um arquivo: **New design file** (em português: **Novo arquivo de design**), com o nome [VitrineCol - protótipo](#). - [] Escolher a ferramenta **Frame** (em português: **Quadro**; tecla F) e, no painel da direita, o tamanho **Phone** (em português: **Celular**). - [] Criar três quadros lado a lado e renomeá-los: [Catálogo](#), [Produto](#) e [Sacola](#). - [] No quadro Catálogo, desenhar o cabeçalho com **Rectangle** (em português: **Retângulo**; tecla R) e **Text** (em português: **Texto**; tecla T): "VitrineCol" e os links "Início", "Catálogo" e "Sacola". - [] Transformar o cabeçalho em componente: botão direito > **Create component** (em português: **Criar componente**; Windows: Ctrl+Alt+K; Mac: Cmd+Option+K). Copiar e colar nos quadros Produto e Sacola. - [] Desenhar o card de produto (retângulo da foto com um X, três linhas: nome, preço e loja), transformá-lo em componente e colocar quatro cópias no Catálogo, em duas colunas. - [] Completar o quadro Produto: foto grande, preço, botões de tamanho (P, M, G), campo de quantidade e botão "Adicionar à sacola". - [] Completar o quadro Sacola: título "Sacola", blocos "Loja A" e "Loja B" (cada um com um item e o subtotal), "Total" e botão "Finalizar sacola". - [] Abrir a aba **Prototype** (em português: **Protótipo**) e ligar o primeiro card ao quadro Produto (**On click**, em português: **Ao clicar**; **Navigate to**, em português: **Navegar para**). - [] Ligar "Adicionar à sacola" ao quadro Sacola e "Finalizar sacola" de volta ao Catálogo. - [] Testar com o botão **Present** (em português: **Apresentar**; o triângulo no canto superior direito) e sair com Esc. - [] Em **Share** (em português: **Compartilhar**), permitir que "qualquer pessoa com o link" possa **view** (em português: **visualizar**) e enviar o link a uma colega. - [] Pedir à colega, sem ajudar, que cumpra a tarefa; anotar onde hesitou e corrigir o protótipo. - [] Colar o link do protótipo no [README.md](#) ou no quadro, para a equipe achar depois. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Importante - **Depende de:** D2-A4 - **Aula e requisitos:** docs/guia-aulas/dia02-aula05-prototipo-navegavel-no-figma.md · não se aplica (planejamento das telas)

Data: 07/10/2026 - quarta-feira - **Cartão (Título no Trello):** D2-A6 – Definir nome, paleta, tipografia e guia de estilo (Marco 1) - **User Story:** Como aluna desenvolvedora, Quero um guia de estilo de uma página com cores, fontes, botão e card, Para que todas as telas tenham a mesma identidade e texto legível (contraste mínimo de 4,5:1). - **Crterios de Aceite:** - O guia de estilo tem: nome e slogan, paleta com código e nome de cada cor, tipografia, botão principal e secundário, card de produto e escala de espaços. - Os 4 pares de contraste testados ([#ffffff](#) sobre [#8a2252](#), [#2b2230](#) sobre [#fbf8f6](#), [#5d5365](#) sobre [#ffffff](#) e [#b3261e](#) sobre [#fde8e6](#)) deram 4,5:1 ou mais, com o resultado anotado. - O protótipo da Aula 5 usa a paleta. - O [README.md](#) no GitHub mostra o link do protótipo e do guia de estilo. - **Marco 1:** protótipo e guia de estilo prontos. - **Checklist interna do cartão:** - [] Usar o nome de trabalho **VitrineCol** (se a equipe trocar, anotar

para trocar nos textos das aulas) e escrever um slogan, por exemplo "Moda das lojas do seu bairro". - [] No Figma, criar um quadro chamado **Guia de estilo**. - [] Desenhar um quadradinho para cada cor, com o código e o nome para o CSS: `--cor-primaria #8a2252`, `--cor-primaria-escura #6b1a40`, `--cor-primaria-clara #f7e6ee`, `--cor-fundo #fbf8f6`, `--cor-superficie #ffffff`, `--cor-texto #2b2230`, `--cor-texto-suave #5d5365`, `--cor-texto-sobre-primaria #ffffff`, `--cor-borda #ddd5da`, `--cor-borda-campo #7a6f82`, `--cor-erro / --cor-erro-fundo`, `--cor-sucesso / --cor-sucesso-fundo`, `--cor-info / --cor-info-fundo`, `--cor-neutro / --cor-neutro-fundo` e `--cor-foco #1a5fb4`. - [] Abrir o verificador de contraste (<https://webaim.org/resources/contrastchecker/>; **Foreground Color** = cor do texto, em português: **Cor do primeiro plano**; **Background Color** = cor do fundo, em português: **Cor de fundo**) e testar os 4 pares da aula. - [] Se a equipe trocar uma cor, repetir o teste; se der menos de 4,5, escurecer o texto até passar. - [] Registrar a fonte do sistema: `system-ui`, `-apple-system`, `"Segoe UI"`, `Roboto`, `"Helvetica Neue"`, `Arial`, `sans-serif`. - [] Registrar os tamanhos: 14 px, 16 px, 18 px, 24 px e 28 px. - [] Desenhar o botão principal (fundo `--cor-primaria`, texto branco, cantos de 12 px, altura mínima de 44 px, negrito) e o secundário (fundo branco, borda e texto na cor primária escura). - [] Desenhar o card de produto (fundo branco, borda fina, foto 4:5, nome em negrito, preço em cor primária escura e loja em texto suave). - [] Registrar a escala de espaços: múltiplos de 4 px (4, 8, 12, 16, 24, 32, 48). - [] Pintar o protótipo da Aula 5 com a paleta (usando os códigos da tabela, sem copiar "a olho"). - [] Colar os links do protótipo e do guia no `README.md` e rodar `git add .`, `git commit -m "..."` e `git push` (se falhar, `git pull` e tentar de novo). - **Etiqueta sugerida:** Front-end (HTML/CSS), Documentação · Importante - **Depende de:** D2·A5 - **Aula e requisitos:** docs/guia-aulas/dia02-aula06-nome-identidade-visual-e-guia-de-estilo.md · não se aplica (identidade visual; prepara contraste e acessibilidade nível AA) · **Marco 1**

Dia 3 · 08/10/2026 · quinta-feira

Entrega do dia: páginas do projeto em HTML, sem estilo (`index.html`, `login.html`, `catalogo.html`, `loja.html` e `produto.html`).

Observações do dia: - A Aula 9 é a mais cheia do dia: seis imagens SVG, três cards no catálogo e duas páginas novas (loja e produto). Se não couber em 60 minutos, as seis imagens (Passo 1) podem ser concluídas em casa antes da Aula 10, sem criar cartão extra. - Hoje não há CSS nem JavaScript; os dados são escritos à mão no HTML. Links para páginas que ainda não existem (Sacola, Cadastrar, Meus pedidos, Painel da loja) são esperados.

Data: 08/10/2026 - quinta-feira - **Cartão (Título no Trello):** D3·A7 – Reescrever o index.html com HTML semântico (cabeçalho e rodapé comuns) - **User Story:** Como aluna desenvolvedora, Quero

uma página com regiões semânticas (header, nav, main, section, footer), Para que leitores de tela e o navegador entendam a estrutura e eu tenha um modelo de cabeçalho e rodapé para copiar nas outras páginas. - **Crítérios de Aceite:** - O `index.html` abre pelo Live Server sem erros e mostra cabeçalho, conteúdo e rodapé. - O painel **Outline** (em português: **Estrutura de tópicos**) mostra um `h1` e dois `h2`. - Apertando Tab, o primeiro foco vai para o link "Ir para o conteúdo". - Toda imagem tem `alt` (busca com Ctrl+F por `<img>`). - **Checklist interna do cartão:** - [] Abrir o `index.html` no Live Server e navegar com a tecla Tab para ver o comportamento antes da mudança. - [] Apagar todo o conteúdo do `index.html` e colar o modelo semântico da aula; salvar com Ctrl+S (Mac: Cmd+S). - [] Conferir no navegador: o menu aparece como lista de links e os links para páginas que ainda não existem (Catálogo, Sacola, Entrar) são esperados. - [] Apertar Tab e conferir a ordem: "Ir para o conteúdo", logotipo e links do menu. - [] Abrir o painel **Outline** (em português: **Estrutura de tópicos**) no Explorer; se não aparecer, usar **View > Open View...** (em português: **Exibir > Abrir Modo de Exibição...**) e escolher **Outline**. Conferir um `h1` e dois `h2`. - [] Experimento: trocar `<main>` por `<div>` (nas duas pontas), ver a região principal sumir no Outline e desfazer com Ctrl+Z (Mac: Cmd+Z). - [] Conferir que o `href` do link de pular e o `id` do `main` têm a mesma grafia, sem acento. - [] Conferir que há um único `h1` na página (o nome do site no cabeçalho é um link `a`). - [] Rodar `git add .`, `git commit -m "..."` (mensagem em português) e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D2-A6 - **Aula e requisitos:** docs/guia-aulas/dia03-aula07-html-semantico.md · não se aplica (base de acessibilidade de todas as telas)

Data: 08/10/2026 - quinta-feira - **Cartão (Título no Trello):** D3-A8 – Criar login.html e catalogo.html com listas e formulários rotulados - **User Story:** Como cliente, Quero um formulário de login e filtros do catálogo claros e com rótulos, Para que eu consiga preencher tudo, inclusive só com o teclado. - **Crítérios de Aceite:** - `login.html` e `catalogo.html` abrem pelo Live Server e mostram os formulários. - Clicar em cada rótulo do login (por exemplo, "E-mail") coloca o cursor no campo certo. - No campo de senha, os caracteres aparecem escondidos. - Todo campo do catálogo tem rótulo: **Buscar pelo nome, Loja, Tamanho** e o texto "Tipo de roupa" para os chips. - Com a tecla Tab, o foco passa por todos os campos e botões do catálogo, na ordem. - **Checklist interna do cartão:** - [] Antes de começar, abrir o wireframe do Login e do Catálogo (Aula 4) e listar os campos de cada um. - [] Criar `login.html` na raiz com o conteúdo da aula (cabeçalho e rodapé copiados do `index.html`). - [] Criar `catalogo.html` na raiz com o conteúdo da aula. - [] Abrir as duas páginas pelo Live Server (**Go Live**, em português: **Ir ao vivo**, ou botão direito > **Open with Live Server**, em português: **Abrir com Live Server**). - [] No login, clicar no texto "E-mail" e conferir que o cursor vai para o campo. - [] Digitar no campo de senha e conferir que os caracteres ficam escondidos. - [] No catálogo, abrir as listas **Loja** e **Tamanho** e escolher uma opção. - [] Conferir que os chips de tipo de roupa são botões comuns (`type="button"`) e a lista de produtos

ainda está vazia. - [] Experimento 1: apagar `for="email"` do primeiro `label`, ver que o clique no texto não foca mais o campo e desfazer. - [] Experimento 2: trocar `type="password"` por `type="text"`, ver a senha visível e desfazer. - [] Conferir que cada `id` é único na página (sem dois `id="email"`). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D3·A7 - **Aula e requisitos:** docs/guia-aulas/dia03-aula08-listas-e-formularios-html.md · só a estrutura de RF-02, RF-03, RF-05 e RF-13

Data: 08/10/2026 - quinta-feira - **Cartão (Título no Trello):** D3·A9 – Montar catalogo, loja, produto e login em HTML e ligar as páginas por links - **User Story:** Como cliente, Quero percorrer catálogo, produto, loja e login só clicando, Para que eu encontre uma peça e saiba de qual loja ela é. - **Critérios de Aceite:** - As quatro páginas (`catalogo.html`, `loja.html`, `produto.html`, `login.html`) abrem pelo Live Server. - O caminho do Passo 5 (Catálogo → Produto → Loja → início → Entrar) funciona só com cliques. - O catálogo mostra 3 cards, a loja mostra 2 e o produto mostra 3 miniaturas e 3 tamanhos (o G desabilitado). - Toda imagem tem `alt` e todo campo tem `label` (busca com Ctrl+F por `<img` e `<input`). - O Outline do VS Code mostra um `h1` em cada página. - **Checklist interna do cartão:** - [] Criar as seis imagens `imagens/exemplo-1.svg` a `imagens/exemplo-6.svg` (uma "camiseta" de cada cor), com os nomes idênticos aos do HTML. - [] No `catalogo.html`, trocar a linha `<ul class="grade-produtos" id="lista-produtos"></ul>` pela lista com três cards. - [] Criar `loja.html` na raiz com o conteúdo da aula (nome, descrição, endereço, botões e dois produtos). - [] Criar `produto.html` na raiz com o conteúdo da aula (foto, miniaturas, preço, loja, descrição, tamanhos, quantidade e botão); conferir que os três botões de tamanho têm o mesmo `name="tamanho"`. - [] Conferir cada página contra o protótipo do Figma (Aula 5). - [] Percorrer o caminho só com cliques: menu **Catálogo** > nome do produto > **Loja Exemplo** > produto > logotipo **VitrineCol** > **Entrar**. - [] Se aparecer "Cannot GET /produto.html", conferir que o arquivo está na raiz, junto do `index.html`. - [] Se uma imagem quebrar, abrir F12 > **Network** (em português: **Rede**) e procurar o erro 404. - [] Confirmar que o cabeçalho não foi colado duas vezes (menu duplicado). - [] Conferir `alt` em toda imagem e `label` em todo campo; abrir o Outline e ver um `h1` por página. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D3·A8 - **Aula e requisitos:** docs/guia-aulas/dia03-aula09-montagem-das-paginas-do-projeto-em-html.md · só a estrutura de RF-01, RF-06, RF-08 e RF-13

Dia 4 · 09/10/2026 · sexta-feira

Entrega do dia: páginas estilizadas conforme o guia de estilo (cores, tipografia, cabeçalho, botões, campos e cards em `css/variaveis.css`, `css/base.css`, `css/componentes.css` e `css/paginas.css`).

Observações do dia: - A Aula 11 (Flexbox) tem quatro partes de CSS para colar e conferir (cabeçalho, botões, campos, avisos e chips). Se faltar tempo, a equipe termina os Passos 3 e 4 em casa antes da Aula 12, sem criar cartão extra. - O catálogo fica em **uma coluna** de propósito; as colunas que mudam com a largura da tela entram no Dia 5.

Data: 09/10/2026 - sexta-feira - **Cartão (Título no Trello):** D4-A10 – Criar `css/variaveis.css` e `css/base.css` e ligar às cinco páginas - **User Story:** Como aluna desenvolvedora, Quero escrever o guia de estilo em código (variáveis, tipografia e reset), Para que trocar uma cor em um só lugar mude todas as páginas. - **Critérios de Aceite:** - As cinco páginas (`index`, `catalogo`, `loja`, `produto`, `login`) carregam sem erro de arquivo CSS no Console (F12 > **Console**). - O fundo é o tom quente claro (`#fbf8f6`), o texto é escuro, os títulos têm tamanhos diferentes e as imagens respeitam a largura da tela. - O conteúdo fica centralizado, com margem nas laterais, em janela larga. - Trocar `--cor-fundo` em `variaveis.css` muda o fundo de todas as páginas de uma vez (e o valor original foi restaurado). - **Checklist interna do cartão:** - [] Criar a pasta `css` na raiz do projeto. - [] Criar `css/variaveis.css` com as cores do guia de estilo, fontes, espaços, bordas arredondadas e sombras. - [] Criar `css/base.css` com o reset leve, a tipografia, os links, as imagens e o contêiner central. - [] Colar as duas linhas `<link rel="stylesheet" href="css/variaveis.css">` e `<link rel="stylesheet" href="css/base.css">` dentro do `<head>`, antes de `</head>`, nas cinco páginas, nessa ordem (variáveis primeiro). - [] Abrir o `catalogo.html` no Live Server e conferir fonte, cores e conteúdo centralizado. - [] Apertar F12, abrir a aba **Elements** (em português: **Elementos**), clicar no título "Catálogo" e achar o diagrama do modelo de caixa na aba **Computed** (em português: **Calculado**); identificar `padding`, `border` e `margin`. - [] Experimento com variáveis: trocar `--cor-fundo: #fbf8f6;` por `--cor-fundo: #ffe9f1;`, ver todas as páginas mudarem e voltar ao valor original. - [] Experimento com seletor: na regra `h1` do `base.css`, trocar `var(--tamanho-titulo)` por `3rem`, ver os títulos crescerem e desfazer (Ctrl+Z; Mac: Cmd+Z). - [] Conferir que `class="container"` está no `<main>` e no `<div>` do cabeçalho. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D3-A9 - **Aula e requisitos:** docs/guia-aulas/dia04-aula10-css-seletores-cores-tipografia-e-caixa.md · não se aplica (identidade visual das telas)

Data: 09/10/2026 - sexta-feira - **Cartão (Título no Trello):** D4-A11 – Montar cabeçalho, menu, botões, campos e chips com Flexbox (`componentes.css`) - **User Story:** Como cliente, Quero um cabeçalho com menu em linha, botões e campos de formulário bem alinhados, Para que eu navegue e preencha formulários sem me perder. - **Critérios de Aceite:** - O cabeçalho mostra o logotipo à esquerda e o menu à direita; em janela estreita, o menu quebra de linha sem rolagem horizontal. - Os botões têm cor, cantos arredondados, altura mínima de 44 px e escurecem ao passar o mouse. - No `login.html`, cada rótulo fica acima do seu campo; no `catalogo.html`, os

chips ficam em linha e o chip "Todas" fica destacado. - A caixa de aviso vermelha de teste apareceu e foi removida do `login.html`. - **Checklist interna do cartão:** - [] Criar `css/componentes.css` com as seções de cabeçalho/menu e de rodapé (Flexbox). - [] Colar `<link rel="stylesheet" href="css/componentes.css">` no `<head>` das cinco páginas, depois do `base.css` (ordem: variáveis, base, componentes). - [] Diminuir a janela do navegador e conferir que o menu quebra de linha sem estourar a tela. - [] Acrescentar no final do arquivo a seção de botões; conferir o botão **Entrar** e os botões da loja e passar o mouse sobre eles. - [] Acrescentar a seção de campos de formulário; conferir o `login.html`, o `catalogo.html` e os botões de tamanho do produto. - [] Acrescentar a seção de avisos e chips; conferir os chips do catálogo em "pílulas". - [] Colar temporariamente `<p class="aviso aviso-erro" role="alert">E-mail ou senha incorretos.</p>` abaixo do `<h1>Entrar</h1>`, ver a caixa vermelha e apagar a linha. - [] Se o menu continuar em coluna, conferir que o `display: flex` está no `ul` (`.menu`) e não no `li` ou no `a`. - [] Conferir que o botão do login tem `class="botao"`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D4-A10 - **Aula e requisitos:** docs/guia-aulas/dia04-aula11-layout-com-flexbox.md · não se aplica (apresentação visual das telas)

Data: 09/10/2026 - sexta-feira - **Cartão (Título no Trello):** D4-A12 – Criar o card de produto com Grid e estilizar o catálogo (`paginas.css`) - **User Story:** Como cliente, Quero ver o catálogo em cards com foto, nome, preço e loja, Para que eu compare as peças de relance e abra a que me interessa. - **Critérios de Aceite:** - O catálogo mostra 3 cards com foto em formato retrato, nome, preço e loja. - O card todo é clicável (abre `produto.html`) e o foco do teclado (Tab) desenha um contorno em volta do card inteiro. - O experimento com `repeat(3, 1fr)` mostrou 3 colunas e o valor voltou para `1fr`. - O formulário de filtros fica dentro de uma caixa branca com borda. - As cinco páginas abrem sem erros no Console. - **Checklist interna do cartão:** - [] Colar a seção do card de produto em `css/componentes.css`, logo antes do comentário `/* ----- Campos de formulário ----- */`. - [] Conferir os cards: foto em retrato, cantos arredondados, sombra, um embaixo do outro (uma coluna). - [] Experimento: na regra `.grade-produtos`, trocar `grid-template-columns: 1fr;` por `repeat(3, 1fr);`, ver 3 colunas e voltar para `1fr`. - [] Criar `css/paginas.css` com os estilos dos filtros do catálogo. - [] Colar `<link rel="stylesheet" href="css/paginas.css">` no `<head>` das cinco páginas, antes de `</head>`. - [] Passar o mouse sobre um card e ver o nome com sublinhado. - [] Apertar Tab até o nome de um produto e ver o contorno em volta do card inteiro. - [] Clicar na foto, no preço e na loja do card e conferir que abre a página do produto. - [] Se os cards tiverem alturas diferentes, conferir `height: 100%` em `.card-produto`; se a foto estiver achatada, conferir `object-fit: cover` e `aspect-ratio` em `.card-produto-imagem`. - [] Abrir o Console (F12 > **Console**) nas cinco páginas e conferir que não há erros. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende**

de: D4-A11 - **Aula e requisitos:** docs/guia-aulas/dia04-aula12-layout-com-grid-e-card-de-produto.md · apresentação visual de RF-01

Dia 5 · 13/10/2026 · terça-feira

Marco: Marco 2 · front-end estático responsivo, versão 0.1 (tag `v0.1`). Evidência: repositório com commits de todas as integrantes.

Entrega do dia: as dez páginas estáticas responsivas, com CSS organizado em quatro arquivos, foco visível e a tag `v0.1`.

Observações do dia: - A Aula 15 é a mais pesada do dia: cinco páginas HTML novas e quatro blocos de CSS para colar. Se não couber em 60 minutos, o Passo 2 (estilos) e a revisão de acessibilidade (Passo 4) são terminados em casa antes da Aula 16; a tag `v0.1` só é criada depois que a revisão estiver toda marcada. - Só uma integrante cria a tag `v0.1`; as outras só conferem no GitHub. - Não há aula em 12/10 (feriado): por isso o Dia 5 acontece em 13/10, logo depois do Dia 4 (09/10).

Data: 13/10/2026 - terça-feira - **Cartão (Título no Trello):** D5-A13 – Tornar catálogo, loja, produto e login responsivos (mobile-first, 768 px e 1024 px) - **User Story:** Como cliente, Quero usar o catálogo, a loja, o produto e o login bem no celular, no tablet e no computador, Para que eu navegue sem rolagem horizontal em qualquer tela. - **Critérios de Aceite:** - Em 360, 768 e 1280 px, nenhuma das quatro páginas tem rolagem horizontal. - O catálogo tem 1 coluna (360 px), 2 colunas (768 px) e 3 ou 4 colunas (1280 px). - A página do produto mostra foto e informações lado a lado a partir de 768 px. - O login aparece como um cartão centralizado. - As cinco páginas têm a linha `<meta name="viewport" ...>` no `<head>`. - **Checklist interna do cartão:** - [] Abrir o `catalogo.html` no Live Server, apertar F12 e clicar em **Toggle device toolbar** (em português: **Alternar barra de ferramentas do dispositivo**; atalho Ctrl+Shift+M, Mac: Cmd+Shift+M). - [] Na barra **Dimensions** (em português: **Dimensões**; ou **Responsive**, em português: **Responsivo**), digitar as larguras 360, 768 e 1280. - [] Colar no final de `css/paginas.css` os estilos da loja, do produto (galeria com miniaturas) e do login. - [] Colar no final de `css/componentes.css` as media queries (seções Tablet e Desktop). - [] Colar no final de `css/paginas.css` as media queries (seção Tablet). - [] Conferir que cada `@media` vem depois das regras que modifica e que todas as chaves `}` estão fechadas. - [] Testar `catalogo.html`, `loja.html`, `produto.html` e `login.html` em 360, 768 e 1280 px, conferindo a tabela da aula (uma coluna a 360; 2 colunas e filtros em 3 colunas a 768; 3 ou 4 colunas e conteúdo centralizado a 1280). - [] Se aparecer rolagem horizontal em 360 px, abrir **Elements** (em português: **Elementos**), achar o elemento que passa da borda e trocar a largura fixa

por `max-width: 100%`. - [] Conferir o `<meta name="viewport" ...>` nas cinco páginas. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D4-A12 - **Aula e requisitos:** docs/guia-aulas/dia05-aula13-responsividade-mobile-first.md · não se aplica (tela funcionando de 360 px a 1280 px, sem rolagem horizontal)

Data: 13/10/2026 - terça-feira - **Cartão (Título no Trello):** D5-A14 – Fechar o base.css: foco visível, link "Ir para o conteúdo", contraste e variáveis - **User Story:** Como cliente, Quero poder usar o site só com o teclado e ler tudo com bom contraste, Para que a plataforma seja acessível a quem não usa o mouse. - **Critérios de Aceite:** - Com a tecla Tab, o primeiro foco mostra "Ir para o conteúdo" e depois todos os elementos interativos mostram um contorno azul de 3 px. - O contraste de dois pares de cores foi anotado, ambos com ✓ (acima de 4,5:1). - Trocar `--cor-primaria` mudou botões, chips, contador e preço ao mesmo tempo, e o valor original foi restaurado. - Os quatro CSS continuam ligados às cinco páginas, na ordem `variaveis`, `base`, `componentes`, `paginas`. - **Checklist interna do cartão:** - [] Sem usar o mouse, abrir o `catalogo.html` e apertar Tab várias vezes para ver como o foco está antes da mudança. - [] Colar no final de `css/base.css` o bloco com foco visível (`:focus-visible`), `.link-pular`, `.visualmente-oculto`, `[hidden]` (com `!important`) e respeito a "menos movimento". - [] Clicar numa área em branco e apertar Tab: o link "Ir para o conteúdo" deve aparecer só quando recebe o foco. - [] Continuar com Tab: logotipo, menu, campos, chips e cards, cada um com contorno azul; no card, o contorno envolve o card inteiro. - [] Abrir F12 > **Elements** (em português: **Elementos**), clicar no nome da loja de um card, e no painel **Styles** (em português: **Estilos**) clicar no quadradinho ao lado de `color`; ler o **Contrast ratio** (em português: **Taxa de contraste**) e conferir o ✓. - [] Repetir no texto branco do botão **Entrar** e anotar os dois valores. - [] Experimento 1: trocar `--cor-primaria: #8a2252;` por `#1d4e89;`, ver tudo mudar junto e voltar ao valor original. - [] Experimento 2: trocar `--cor-foco: #1a5fb4;` por `#ff6600;`, apertar Tab, ver o contorno laranja e voltar ao valor original. - [] Conferir a grafia idêntica de `visualmente-oculto` no HTML e no CSS e usar `:focus-visible` (não `:focus`). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS) · Essencial - **Depende de:** D5-A13 - **Aula e requisitos:** docs/guia-aulas/dia05-aula14-variaveis-css-hover-e-foco-visivel.md · não se aplica (acessibilidade nível AA básico: foco visível, contraste, menos movimento)

Data: 13/10/2026 - terça-feira - **Cartão (Título no Trello):** D5-A15 – Criar as cinco telas estáticas restantes, revisar a acessibilidade e marcar a versão 0.1 (Marco 2) - **User Story:** Como lojista, Quero as telas de cadastro, sacola, minha loja, meus produtos e formulário de produto prontas e acessíveis, Para que a equipe possa ligá-las ao JavaScript e ao banco nos próximos dias. - **Critérios de Aceite:** - As dez páginas (`index`, `catalogo`, `loja`, `produto`, `login`, `cadastro`, `sacola`,

`painel-loja` , `painel-produtos` , `painel-produto-form`) abrem pelo Live Server e são navegáveis pelo menu e pelos links. - A tabela do `painel-produtos.html` vira blocos em 360 px e volta a ser tabela em 768 px. - A lista de acessibilidade está toda marcada: um único `h1` por página, `alt` em toda imagem, `label` em todo campo, foco visível, `type="button"` nos botões que não enviam, preços como `R$ 129,90` e sem rolagem horizontal em 360 e 1280 px. - `git shortlog -sn` mostra commits de todas as integrantes e a tag `v0.1` aparece no GitHub (aba **Tags**). - **Marco 2:** front-end estático responsivo, versão 0.1. - **Checklist interna do cartão:** - [] Abrir o quadro e arrastar os cartões ligados a RF-12, RF-15, RF-16 e RF-17 para "Em Andamento" (um cartão por dupla). - [] Criar na raiz `cadastro.html` , `sacola.html` , `painel-loja.html` , `painel-produtos.html` e `painel-produto-form.html` com o conteúdo da aula; salvar com Ctrl+S (Mac: Cmd+S). - [] Colar em `css/componentes.css` , logo antes de `/* ----- Tablet ----- */` , as seções de selos de status do pedido e de tabela. - [] Colar em `css/paginas.css` , logo antes de `/* ----- Login e cadastro ----- */` , a seção da sacola. - [] Colar em `css/paginas.css` , logo antes de `/* ----- Tablet ----- */` , a seção do painel da lojista. - [] Colar no final de `css/paginas.css` a seção do formulário do produto. - [] Abrir cada página pelo Live Server e conferir: cadastro (nome, e-mail, senha, telefone e escolha **Cliente/Lojista**), sacola (dois blocos de loja, subtotais, total e botão "Finalizar sacola (2 pedidos)"), painel da loja, tabela de produtos (blocos em 360 px) e formulário do produto (uma linha de tamanho e estoque, sem fotos ainda). - [] Nas dez páginas, usar Ctrl+F por `<img` , `<input` e `<h1` e marcar a lista: um `h1` ; `alt` ; `label` com `for` e `id` ; foco com Tab; `type="button"` ; preços com vírgula; sem rolagem horizontal em 360 e 1280 px. - [] Corrigir o que a lista apontar e comentar com a equipe. - [] Rodar `git add .` , `git commit -m "..."` e `git push` . - [] Rodar `git shortlog -sn` e conferir que todas as integrantes aparecem. - [] (Só uma integrante) Criar e enviar a tag `v0.1` (`git tag v0.1` e `git push origin v0.1`); as outras conferem na aba **Tags** do GitHub. - [] Arrastar o cartão do Trello para "Em Revisão" só depois do push e da tag. - **Etiqueta sugerida:** Front-end (HTML/CSS), Testes · Essencial - **Depende de:** D5-A14 - **Aula e requisitos:** docs/guia-aulas/dia05-aula15-revisao-e-fechamento-do-front-end-estatico.md · só a estrutura de RF-09, RF-12, RF-15, RF-16, RF-17 e RF-19 · **Marco 2**

Dia 6 · 14/10/2026 · quarta-feira

Entrega do dia: interações básicas nas páginas (script ligado ao catálogo, produtos fictícios como array de objetos, cabeçalho gerado pelo JavaScript, avisos e chips que reagem ao clique).

Observações do dia: - Os exercícios opcionais das Aulas 16 e 17 (por exemplo, o desafio "Últimas unidades!", a função `calcularDesconto` e o 7º produto) são o primeiro trabalho que sobra se faltar

tempo; não viram cartão extra. - Só o `catalogo.html` passa a gerar o cabeçalho pelo JavaScript hoje; as outras páginas mantêm o cabeçalho escrito à mão por enquanto.

Data: 14/10/2026 - quarta-feira - **Cartão (Título no Trello):** D6-A16 – Ligar o primeiro script ao catálogo e treinar variáveis, tipos e condicionais no Console - **User Story:** Como aluna desenvolvedora, Quero ligar um script JavaScript ao catálogo e ver mensagens no Console, Para que eu entenda variáveis, tipos, operadores e a regra "preço maior que zero". - **Critérios de Aceite:** - O Console (F12 > **Console**) mostra as 6 mensagens do Passo 3 (nome, preço, tipo `number`, "disponível", "Preço válido." e o preço com desconto `116.91`), sem erros vermelhos. - Com `estoque = 0` aparece "esgotado"; com `preco = -5` aparece "Preço inválido". - Tentar mudar uma `const` gera o erro `Assignment to constant variable.` e a linha de teste foi apagada. - A condicional do desafio mostra "Últimas unidades!" com estoque 1 ou 2. - **Checklist interna do cartão:** - [] Criar as pastas `js` e `js/paginas` (no **Explorer**, em português: **Explorador**, usar **New Folder**, em português: **Nova Pasta**). - [] Criar `js/paginas/catalogo.js` com o conteúdo da aula. - [] No `catalogo.html`, colar `<script type="module" src="js/paginas/catalogo.js"></script>` logo antes de `</body>`. - [] Abrir pelo **Go Live** (em português: **Ir ao vivo**), nunca direto do arquivo (`file://`), e abrir F12 > **Console**. - [] Conferir as 6 mensagens esperadas. - [] Exercício 1: trocar `const estoque = 3;` por `0` e ver "esgotado". - [] Exercício 2: trocar `let preco = 129.9;` por `-5` e ver "Preço inválido"; voltar o valor. - [] Exercício 3: acrescentar `preco = 139.9;` abaixo da linha do preço e ver o novo valor. - [] Exercício 4: escrever `estoque = 1;` (que é `const`), ver o erro vermelho `Assignment to constant variable.` e apagar a linha. - [] Desafio: escrever uma condicional que mostre "Últimas unidades!" quando o estoque for maior que zero e menor que 3 (usar `&&`). - [] Se nada aparecer, conferir o `src` do script e, em F12 > **Network** (em português: **Rede**), se há erro 404. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript · Essencial - **Depende de:** D5-A15 - **Aula e requisitos:** docs/guia-aulas/dia06-aula16-javascript-variaveis-tipos-e-condicionais.md · não se aplica (base de programação; ensaia a RN-02)

Data: 14/10/2026 - quarta-feira - **Cartão (Título no Trello):** D6-A17 – Representar produtos como array de objetos e escrever funções que os percorrem - **User Story:** Como aluna desenvolvedora, Quero guardar os produtos fictícios em uma lista de objetos e percorrê-la com funções, Para que o catálogo seja desenhado a partir de dados e não escrito à mão. - **Critérios de Aceite:** - O Console mostra 6 linhas de resumo (uma por produto, com o estoque total), `false` para "Vestido tem estoque no G?" e `99.80` para o subtotal de duas camisetas. - A função `calcularDesconto(100, 10)` devolve `90`. - Um 7º produto acrescentado à lista aparece no laço sem mudar o código do laço. - Não há erros vermelhos no Console. - **Checklist interna do cartão:** - [] Abrir o `catalogo.js` e confirmar que ainda mostra as mensagens da Aula 16. - [] Apagar todo o

conteúdo de `js/paginas/catalogo.js` e colar a versão da aula, com a lista `produtosFicticios` (`id`, `nome`, `descricao`, `preco`, `categoria`, `lojaId`, `lojaNome`, `tamanhos`, `fotos`) e as funções que a percorrem com `for...of`. - [] Abrir o Console e conferir as 6 linhas, o `false` do vestido no G e o `99.80`. - [] Exercício 1: mudar o `estoque` de G do primeiro produto para `2` e ver a resposta passar a `true`. - [] Exercício 2: escrever `calcularDesconto(preco, percentual)` com `return` e testar `console.log(calcularDesconto(129.9, 20))`. - [] Exercício 3: acrescentar um 7º produto com os mesmos campos (foto `exemplo-1.svg`) e conferir que o laço o mostra. - [] Exercício 4: mostrar só o nome da última peça com `console.log(produtosFicticios[produtosFicticios.length - 1].nome)`. - [] Se aparecer `Cannot read properties of undefined`, lembrar que a contagem das posições começa em 0. - [] Conferir as vírgulas entre os objetos e as chaves `}` de cada um. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript · Essencial - **Depende de:** D6-A16 - **Aula e requisitos:** docs/guia-aulas/dia06-aula17-funcoes-arrays-e-objetos.md · não se aplica (dados fictícios; ensaia a RN-02)

Data: 14/10/2026 - quarta-feira - **Cartão (Título no Trello):** D6-A18 – Mexer na página com o DOM: criarElemento, avisos, cabeçalho em JavaScript e chips clicáveis - **User Story:** Como cliente, Quero ver avisos de "Carregando...", erro e sucesso e chips que reagem ao clique, Para que a tela nunca fique em branco ou sem resposta. - **Critérios de Aceite:** - O menu aparece gerado pelo JavaScript e o link **Catálogo** está destacado como página atual. - "Carregando..." aparece abaixo do título e some sozinho depois de cerca de 1,5 segundo. - Ao clicar em um chip (por exemplo, **Vestidos**), ele fica pressionado e aparece "Você escolheu: Vestidos"; ao clicar em outro, o anterior volta ao normal. - O teste `document.querySelector("h1").textContent = "<b>teste</b>"` mostra o texto `<b>teste</b>` literalmente, sem negrito. - Procurando `innerHTML` em `js/` (Ctrl+Shift+F; Mac: Cmd+Shift+F) não há resultados, e o Console não mostra erros. - **Checklist interna do cartão:** - [] Criar a pasta `js/ui` e o arquivo `js/ui/elementos.js` com a função `criarElemento`. - [] Criar `js/ui/avisos.js` (avisos de carregamento, erro, sucesso e informação, usando as classes `aviso`). - [] Criar `js/ui/cabecalho.js`, que gera o cabeçalho comum. - [] No `catalogo.html`, trocar o bloco `<header class="cabecalho" id="cabecalho">` inteiro por `<header class="cabecalho" id="cabecalho"></header>`. - [] Trocar o conteúdo de `js/paginas/catalogo.js` pela versão da aula (mantém os produtos fictícios e acrescenta avisos e eventos de clique nos chips). - [] Abrir o `catalogo.html` no Live Server e conferir o menu, o "Carregando..." que some e o chip pressionado. - [] Teste de segurança no Console (F12 > **Console**): rodar `document.querySelector("h1").textContent = "<b>teste</b>"`, ver o texto sem negrito e recarregar a página. - [] Procurar `innerHTML` em `js/` e conferir que não há resultados. - [] Se o cabeçalho aparecer duplicado, conferir que o `<header>` do HTML ficou vazio; se der 404 no `import`, conferir o caminho `../ui/cabecalho.js`. - [] Conferir que o `id="chips-categorias"` do

HTML é igual ao usado no `getElementById` . - [] Rodar `git add .` , `git commit -m "..."` e `git push` . - **Etiqueta sugerida:** JavaScript, Segurança · Essencial - **Depende de:** D6·A17 - **Aula e requisitos:** docs/guia-aulas/dia06-aula18-dom-e-eventos.md · início de RF-21

Dia 7 · 15/10/2026 · quinta-feira

Entrega do dia: catálogo filtrável com dados fictícios (cards desenhados pelo JavaScript, filtros por tipo, loja, tamanho e nome) e formulário de produto com validação.

Observações do dia: - A Aula 20 é a que mais pesa: são cinco funções novas coladas em ordem no `catalogo.js` . Se faltar tempo, o que sobrar do teste final (Passo 6, itens 4 e 5, tamanho e busca por nome) é concluído antes da Aula 21, sem criar cartão extra. - A busca pelo nome (RF-05) é Desejável; aqui ela é só ensaiada com dados fictícios. A versão com o banco vem no Dia 10. - O 15/10 foi tratado como dia normal de aula.

Data: 15/10/2026 - quinta-feira - **Cartão (Título no Trello):** D7·A19 – Desenhar o catálogo pelo JavaScript: cards, preço em R\$ e estado vazio - **User Story:** Como cliente, Quero ver os produtos em cards com foto, nome, preço em reais e loja, Para que eu compare as peças e nunca veja uma tela em branco. - **Critérios de Aceite:** - O catálogo mostra "Carregando..." por cerca de 0,6 segundo e depois 6 cards, cada um com foto, nome, preço no formato `R$ 129,90` e loja. - Com a lista vazia (`mostrarProdutos([])`), aparece "Nenhum produto encontrado." em vez de uma tela em branco. - Um 7º produto acrescentado à lista aparece sozinho como novo card. - O Console não mostra erros e a busca por `innerHTML` em `js/` (Ctrl+Shift+F; Mac: Cmd+Shift+F) dá 0 resultados. - **Checklist interna do cartão:** - [] Criar `js/ui/formatadores.js` com a função `formatarPreco` (usa `Intl.NumberFormat` em R\$), com a palavra `export` na frente. - [] No `js/paginas/catalogo.js` , trocar o começo do arquivo (até a linha de `import` de `aviso.js`) pelo trecho da aula. - [] Manter a lista `produtosFicticios` e trocar tudo o que vem depois dela (a partir de `const listaChips = ...`) pelo código da aula, que cria cada card com `createElement` e `textContent` . - [] No `catalogo.html` , trocar a lista `<ul class="grade-produtos" id="lista-produtos">` inteira por `<ul class="grade-produtos" id="lista-produtos"></ul>` (sem os 3 cards escritos à mão) e conferir o aviso de lista vazia (`id="mensagem-vazio"`). - [] Abrir o catálogo e conferir "Carregando..." e os 6 cards com preço `R$ 129,90` . - [] Clicar em um card e conferir que abre a página do produto. - [] Teste do estado vazio: trocar `mostrarProdutos(produtosFicticios);` por `mostrarProdutos([]);` , ver "Nenhum produto encontrado." e voltar ao código original. - [] Acrescentar um 7º produto à lista e ver o 7º card aparecer. - [] Procurar `innerHTML` em `js/` e confirmar 0 resultados. - [] Se os cards antigos aparecerem junto com os novos, conferir que a `ul` do HTML está vazia. - [] Rodar `git add .` ,

`git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript, Front-end (HTML/CSS) · Essencial - **Depende de:** D6·A18 - **Aula e requisitos:** docs/guia-aulas/dia07-aula19-catalogo-a-partir-de-um-array-de-objetos.md · RF-01 (com dados fictícios), RF-21

Data: 15/10/2026 - quinta-feira - **Cartão (Título no Trello):** D7·A20 – Filtrar o catálogo por tipo, loja, tamanho e nome com filter, find e includes - **User Story:** Como cliente, Quero filtrar as roupas por tipo, loja e tamanho e buscar pelo nome, Para que eu encontre rápido o que procuro. - **Critérios de Aceite:** - Os 8 chips de tipo de roupa e as 2 lojas aparecem sozinhos (criados pelo JavaScript); o chip **Todas** vem marcado. - Cada filtro funciona sozinho e combinado com os outros; **Todas** limpa o filtro de tipo. - Escolhendo o tamanho **G**, o vestido (estoque 0 em G) não aparece e as camisetas aparecem. - Digitar `calça` ou `CAMISETA` encontra o produto, sem diferenciar maiúsculas de minúsculas. - Quando nada combina (por exemplo, **Vestidos + Loja do Bairro**), aparece "Nenhum produto encontrado." - **Checklist interna do cartão:** - [] No `catalogo.html`, trocar o formulário de filtros inteiro (de `<form class="filtros" aria-label="Filtros do catálogo">` até `</form>`) pela versão da aula, com só o chip **Todas** e a opção **Todas**. - [] No `catalogo.js`, trocar as constantes `listaProdutos` e `mensagemVazio` pelo bloco da aula (campos do formulário, dados fictícios de lojas e tipos e o objeto `filtros`). - [] Colar `temEstoque` e `filtrarProdutos` logo antes de `criarCardProduto`. - [] Colar `atualizarCatalogo`, `preencherCategorias`, `preencherLojas`, `escolherCategoria` e `configurarFiltros` logo antes de `iniciar`, nessa ordem. - [] Trocar a função `iniciar` e a chamada `iniciar();` do final pela versão da aula. - [] Conferir os `id` do HTML: `busca`, `filtro-loja`, `filtro-tamanho` e `chips-categorias`. - [] Testar: **Vestidos** mostra só o vestido; **Todas** traz os 6 de volta. - [] Testar: **Loja do Bairro** mostra só os produtos dela; **Vestidos + Loja do Bairro** mostra "Nenhum produto encontrado." - [] Testar: tamanho **G** esconde o vestido; busca por `calça` e por `CAMISETA`. - [] Se o filtro de loja não mostrar nada, conferir que `opcao.value = loja.id` (e não o nome). - [] Se algo falhar, escrever `console.log(filtros);` dentro de `atualizarCatalogo` e ver o estado no Console (F12 > **Console**). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript · Essencial - **Depende de:** D7·A19 - **Aula e requisitos:** docs/guia-aulas/dia07-aula20-filtros-com-metodos-de-array.md · RF-02, RF-03, RF-04 (dados fictícios); ensaia RF-05

Data: 15/10/2026 - quinta-feira - **Cartão (Título no Trello):** D7·A21 – Validar o formulário de produto no navegador (submit, preventDefault e erros ao lado do campo) - **User Story:** Como lojista, Quero que o formulário de produto me avise de cada erro ao lado do campo, Para que eu corrija preço, estoque e telefone antes de salvar. - **Critérios de Aceite:** - Salvar o formulário vazio mostra uma mensagem vermelha ao lado de cada campo inválido ("Informe o nome do produto.", "Escolha a categoria.", "O preço deve ser maior que zero.", "Informe o tamanho.") e leva o foco ao primeiro. - Preço `-5` e estoque `-1` são recusados com mensagens claras; preço `129,90` (com

vírgula) é aceito. - Com tudo certo aparece **Tudo certo!** e a página não recarrega. - No Console, `telefoneValido(normalizarTelefone("(27) 99999-9999"))` é `true` (RN-10). - **Checklist interna do cartão:** - [] No final de `js/ui/formatadores.js`, acrescentar `normalizarTelefone` e `telefoneValido` (regra do telefone escrita uma única vez). - [] No final de `js/ui/avisos.js`, colar as funções que mostram e apagam o erro de um campo. - [] No final de `css/componentes.css`, colar o estilo de campo com erro. - [] Criar a pasta `js/servicos` e o arquivo `js/servicos/produtoServico.js`, por enquanto só com a validação do produto (preço maior que zero, estoque não negativo; RN-02). - [] Criar `js/paginas/painelProdutoForm.js`, que trata o `submit` com `evento.preventDefault()`, limpa os erros **antes** de validar e mostra a mensagem "Tudo certo!". - [] No `painel-produto-form.html`, trocar o bloco `<header>` inteiro por `<header class="cabecalho" id="cabecalho"></header>` e colar `<script type="module" src="js/paginas/painelProdutoForm.js"></script>` antes de `</body>`. - [] Abrir `painel-produto-form.html` e clicar em **Salvar produto** sem preencher nada; conferir os 4 erros e o foco no primeiro campo. - [] Testar preço `-5` e estoque `-1`; depois preço `129,90` e estoque `3` e conferir "Tudo certo!" sem recarregar. - [] No Console (F12 > **Console**), colar `const f = await import("/js/ui/formatadores.js");` e testar o telefone; esperado `5527999999999`, `true` e `false`. - [] Conferir que cada campo está dentro de um `<div class="campo">`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript, Front-end (HTML/CSS) · Essencial - **Depende de:** D7-A20 - **Aula e requisitos:** docs/guia-aulas/dia07-aula21-formularios-e-validacao-no-navegador.md · RN-02, RN-10 (telefone), RF-21

Dia 8 · 16/10/2026 · sexta-feira

Entrega do dia: classes de domínio do projeto (`ErroApp`, `Produto`, `Loja`, `Usuaría`, `Cliente`, `Lojista`), catálogo com dados fictícios usando as classes, tratamento de erros e código em módulos (`modelos`, `ui`, `paginas`).

Observações do dia: - A Aula 22 tem a maior sequência de trocas no `catalogo.js` (oito substituições, na ordem). Se faltar tempo, o teste do Passo 5 (item 3, no Console) é concluído antes da Aula 23, sem cartão extra. - Com a Aula 24 termina o Módulo 2 (JavaScript e orientação a objetos). Os dados continuam fictícios; o banco só entra no Dia 9.

Data: 16/10/2026 - sexta-feira - **Cartão (Título no Trello):** D8-A22 – Criar ErroApp, Produto e Loja com campos privados e regras dentro da classe - **User Story:** Como lojista, Quero que o sistema recuse produtos com preço zero, estoque negativo ou mais de 5 fotos, Para que nenhum produto inválido entre no catálogo. - **Critérios de Aceite:** - O catálogo funciona como antes, agora usando objetos `Produto` e `Loja`. - Com preço `0` no primeiro produto, o Console mostra `0 preço deve`

ser maior que zero. e nenhum card aparece; depois o preço volta para 129.9 . - No Console, `p.preco = -1` lança o erro e o preço continua 10 (RN-02). - Ler um campo privado de fora (`p.#preco`) dá erro de sintaxe: o encapsulamento funciona. - O produto sem foto devolve `imagens/sem-foto.svg` como capa (RN-12) e o WhatsApp da loja só aceita dígitos com país e DDD (RN-10). - **Checklist interna do cartão:** - [] Criar a pasta `js/modelos` e o arquivo `js/modelos/ErroApp.js` (erro próprio que herda de `Error`, com `export`). - [] Criar `js/modelos/Produto.js` com campos privados `#`, getters, `temEstoque`, `precoFormatado()`, `fotoCapa()` e as regras de preço, estoque e fotos. - [] Criar `js/modelos/Loja.js` com a lista privada de produtos e a regra do WhatsApp. - [] No `catalogo.js`, fazer as trocas na ordem da aula: (1) as duas linhas de `import` de `Produto` e `Loja` no lugar de `formatarPreco`; (2) lojas e produtos viram objetos `Loja` e `Produto`; (3) apagar a função `temEstoque`; (4) usar `produto.temEstoque(filtros.tamanho)`, `produto.fotoCapa()`, `produto.precoFormatado()`, `filtrarProdutos(produtos, filtros)` e `preencherLojas(lojas)`. - [] Abrir o catálogo: filtros, preços em R\$ e estado vazio devem funcionar igual à Aula 21. - [] Teste da regra: trocar o preço do primeiro produto por 0, recarregar, ver o erro `O preço deve ser maior que zero.` no Console (F12 > **Console**) e voltar para 129.9 . - [] No Console, colar o teste da aula (`const { Produto } = await import("/js/modelos/Produto.js");`) e conferir `Teste R$ 10,00 true false imagens/sem-foto.svg`, depois `preco_invalido - O preço deve ser maior que zero.` e por fim 10 . - [] Conferir que `new Produto({...})` usa `new` e que `export class` está nos três arquivos. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Orientação a Objetos, JavaScript · Essencial - **Depende de:** D7-A21 - **Aula e requisitos:** docs/guia-aulas/dia08-aula22-classes-javascript-produto-e-loja.md · RN-02, RN-10, RN-12

Data: 16/10/2026 - sexta-feira - **Cartão (Título no Trello):** D8-A23 – Criar Usuaría, Cliente e Lojista (herança e polimorfismo) e ligar Loja a Produto (agregação) - **User Story:** Como aluna desenvolvedora, Quero que cliente e lojista herdem da mesma classe e cada uma responda a sua página inicial, Para que o código não se repita e o sistema saiba para onde levar cada perfil. - **Crítérios de Aceite:** - No Console, `rotaInicial()` devolve `index.html` para a cliente e `painel-loja.html` para a lojista (mesma chamada, respostas diferentes). - `cliente instanceof Usuaría` é `true` e `cliente instanceof Lojista` é `false`. - Chamar `rotaInicial()` em uma `Usuaría` pura mostra o erro `metodo_nao_implementado`. - O catálogo funciona como antes, e um produto com `ativo: false` não aparece (RN-08). - **Checklist interna do cartão:** - [] Criar `js/modelos/Usuaría.js`, `js/modelos/Cliente.js` e `js/modelos/Lojista.js` (as filhas não têm construtor e usam o da mãe). - [] Abrir o `catalogo.html` pelo Live Server, abrir F12 > **Console** e colar o teste da aula (`const { Usuaría } = await import("/js/modelos/Usuaría.js");`). Esperado: duas rotas diferentes, `true true false` e `metodo_nao_implementado`. - [] No `catalogo.js`, trocar a linha `const produtos = produtosFicticios.map(...)` pela versão que adiciona cada produto à

lista da sua loja (agregação). - [] Trocar a função `atualizarCatalogo` pela versão que lista só os produtos **ativos** de cada loja. - [] Teste de RN-08: acrescentar `ativo: false`, ao produto "Saia plissada", ver a saia sumir e tirar a linha para ela voltar. - [] Teste de erro: trocar o `lojaId` de um produto por `"19"`, recarregar, ver o erro no Console e desfazer. - [] Conferir que `export class Usuaría` tem o `export`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Orientação a Objetos · Essencial - **Depende de:** D8·A22 - **Aula e requisitos:** docs/guia-aulas/dia08-aula23-heranca-polimorfismo-e-agregacao.md · RN-01, RN-08, RN-11

Data: 16/10/2026 - sexta-feira - **Cartão (Título no Trello):** D8·A24 – Tratar erros com try/catch e ErroApp e organizar o código em módulos - **User Story:** Como cliente, Quero ver uma mensagem clara em português quando algo der errado, Para que eu nunca fique diante de uma tela em branco. - **Crterios de Aceite:** - O catálogo continua funcionando (filtros, preços e estado vazio). - Com preço `0` em um produto, aparece uma caixa vermelha com "O preço deve ser maior que zero." e a tela não fica em branco. - Com um erro de programação (por exemplo, `lojasX.find`), aparece "Algo deu errado. Recarregue a página e tente novamente." e o Console mostra o detalhe técnico. - `catalogo.js` não tem mais a função do card; `js/ui/cards.js` existe. - A busca por `innerHTML` em `js/` tem 0 resultados. - **Checklist interna do cartão:** - [] Criar `js/ui/cards.js` com a função `criarCardProduto` exportada (monta o card só com `createElement` e `textContent`). - [] No `js/ui/aviso.js`, colocar o `import` do `ErroApp` logo após o primeiro comentário e colar as funções `mensagemDoErro` e `registrarErro` antes da seção "Erros ao lado de cada campo de formulário". - [] No `catalogo.js`, trocar as importações para trazer o card de `cards.js` e as funções de erro de `aviso.js`. - [] Tirar a criação das lojas e dos produtos do corpo do arquivo e apagar a função `criarCardProduto` do catálogo. - [] Colar a função `carregarLojasFicticias` antes de `iniciar` e trocar `iniciar` pela versão com `try/catch`, chamando `mostrarErro(mensagemDoErro(erro))`. - [] Teste 1: abrir o catálogo e conferir que funciona como antes. - [] Teste 2: preço `0` no primeiro produto, recarregar, ver a caixa vermelha e voltar para `129.9`. - [] Teste 3: trocar `lojas.find(...)` por `lojasX.find(...)`, recarregar, ver a mensagem geral, olhar o Console (F12 > **Console**) e desfazer. - [] Conferir a organização em `js/modelos`, `js/ui` e `js/paginas`. - [] Procurar `innerHTML` em `js/` (Ctrl+Shift+F; Mac: Cmd+Shift+F) e confirmar 0 resultados. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript, Orientação a Objetos · Essencial - **Depende de:** D8·A23 - **Aula e requisitos:** docs/guia-aulas/dia08-aula24-tratamento-de-erros-e-modulos.md · RF-21

Dia 9 · 19/10/2026 · segunda-feira

Entrega do dia: banco criado e populado no Supabase (8 tabelas, 4 gatilhos, 8 categorias, lojista de teste, uma loja e 3 produtos de exemplo).

Observações do dia: - **Regra para os Dias 9 a 12:** a RLS (segurança por linha) fica **desativada**, a política provisória de Storage entra no Dia 11 e a lojista de teste é quem faz as ações (ainda não há login). Quem ativar a RLS antes do Dia 13 trava as telas dos dias seguintes. A RLS definitiva e o login só entram no Dia 13. - A Aula 26 tem o script mais comprido do curso (`01_schema.sql`). Copiar o script **inteiro**, sem cortar linhas; se der erro no meio, seguir a seção "Se travar" da aula e não pular para a Aula 27 sem as 8 tabelas e os 4 gatilhos. - Passos de **banco** (scripts SQL) nunca são pulados: as aulas seguintes dependem deles. - A senha do banco é guardada **fora** do repositório (nunca no código nem no GitHub). Regra do plano gratuito: entrar no painel ou rodar o site pelo menos uma vez por semana, para o projeto não ser pausado.

Data: 19/10/2026 - segunda-feira - **Cartão (Título no Trello):** D9-A25 – Criar o projeto vitrine-col no Supabase e ler o diagrama das 8 tabelas - **User Story:** Como aluna desenvolvedora, Quero criar o banco em nuvem da equipe e entender as 8 tabelas e suas ligações "1 para N", Para que os dados da plataforma fiquem guardados de verdade e não somem a cada recarga. - **Críticos de Aceite:** - O projeto `vitrine-col` aparece ativo no painel do Supabase, sem mensagem de erro ou "projeto pausado". - A senha do banco está guardada em lugar seguro **fora** do repositório. - Olhando o diagrama, a equipe diz que existem 8 tabelas e o que significa `1 : N`. - As respostas das 5 perguntas de leitura foram conferidas pela equipe (3 lojas geram 3 pedidos; um produto tem no máximo 5 fotos; apagar a loja apaga os produtos em cascata; `tamanhos` é separada de `produtos` por ser relação 1:N; `itens_pedido` liga pedidos a produtos). - **Checklist interna do cartão:** - [] (Uma integrante) Entrar em <https://supabase.com> e clicar em **Start your project** (em português: **Comece o seu projeto**), escolher **Continue with GitHub** (em português: **Continuar com o GitHub**) e autorizar. - [] Se o painel pedir, criar uma organização com o nome da equipe e escolher o plano **Free** (em português: **Gratuito**). - [] Clicar em **New project** (em português: **Novo projeto**) e preencher **Name** (em português: **Nome**): `vitrine-col`. - [] Em **Database Password** (em português: **Senha do banco**), clicar em **Generate a password** (em português: **Gerar uma senha**) e guardar a senha fora do repositório. - [] Em **Region** (em português: **Região**), escolher **South America (São Paulo)** (em português: **América do Sul (São Paulo)**). - [] Clicar em **Create new project** (em português: **Criar novo projeto**) e esperar cerca de 2 minutos até o painel carregar. - [] Convidar as outras integrantes por e-mail em **Team** (em português: **Equipe**) > **Invite** (em português: **Convidar**). - [] Identificar no menu da esquerda, sem modificar nada: **Table Editor** (em português: **Editor de tabelas**), **SQL Editor** (em português: **Editor SQL**), **Authentication** (em português: **Autenticação**), **Storage** (em português: **Armazenamento**) e **Project Settings** (em português: **Configurações do projeto**). - [] Abrir <https://supabase.com/pricing>, anotar os limites do plano gratuito (cerca de 500 MB de banco, 1 GB de arquivos, poucos e-mails por hora) e a regra de pausa por falta de uso; registrar no quadro do Trello o combinado de entrar no painel toda semana. - [] Ler a lista das 8 tabelas (`perfis` , `lojas` , `categorias` , `produtos` , `tamanhos` ,

`produto_fotos` , `pedidos` , `itens_pedido`) e o diagrama, e treinar a leitura de `lojas 1 : N produtos` e `perfis 1 : 0..1 lojas` . - [] Responder às 5 perguntas de leitura no caderno e conferir com a equipe. - **Etiqueta sugerida:** Supabase (Banco de Dados), Configuração · Essencial - **Depende de:** D8-A24 - **Aula e requisitos:** docs/guia-aulas/dia09-aula25-supabase-e-modelagem-do-banco.md · base de RN-01, RN-03 e RN-12

Data: 19/10/2026 - segunda-feira - **Cartão (Título no Trello):** D9-A26 – Rodar o 01_schema.sql no Supabase: 8 tabelas, 4 gatilhos e 8 categorias - **User Story:** Como aluna desenvolvedora, Quero criar as 8 tabelas com chaves, regras e gatilhos usando SQL, Para que o banco recuse preço zero, mais de 5 fotos e outras entradas inválidas. - **Critérios de Aceite:** - A consulta de tabelas em `information_schema.tables` devolve exatamente 8 linhas (`categorias` , `itens_pedido` , `lojas` , `pedidos` , `perfis` , `produto_fotos` , `produtos` , `tamanhos`). - A consulta de gatilhos devolve 4 linhas (`ao_criar_usuario` , `fotos_limite_por_produto` , `perfis_ao_trocar_tipo` e `pedidos_protegidos`). - A consulta de `CHECK` lista as regras de preço, estoque, ordem, quantidade, status e tipo. - `select id, nome from public.categorias order by id;` devolve 8 linhas: Blusas, Camisetas, Vestidos, Calças, Saias, Shorts, Jaquetas e Acessórios. - No **Table Editor** (em português: **Editor de tabelas**) aparecem as 8 tabelas, com a RLS **desligada**. - **Checklist interna do cartão:** - [] No **Table Editor** (em português: **Editor de tabelas**), confirmar que ainda não há tabelas. - [] Criar a pasta `database` na raiz do projeto e o arquivo `database/01_schema.sql` com a Parte 1 (as 8 tabelas e os índices). - [] No painel, abrir **SQL Editor** (em português: **Editor SQL**) > **New query** (em português: **Nova consulta**), colar a Parte 1 (do primeiro `create table` ao último `create index`) e clicar em **Run** (em português: **Executar**). - [] Se o painel avisar sobre RLS, escolher a opção sem RLS (**Run without RLS**, em português: **Executar sem RLS**); a segurança por linha só entra no Dia 13. - [] Conferir a mensagem **Success. No rows returned** (em português: **Sucesso. Nenhuma linha retornada**) e rodar a consulta que lista as 8 tabelas. - [] Colar a Parte 2 (quatro gatilhos e a função usada pela cliente) no **final** do `database/01_schema.sql` . - [] Em uma nova consulta, rodar a Parte 2 (do comentário `-- Gatilho: cria o perfil...` até o fim) e rodar a consulta que lista os 4 gatilhos. - [] Rodar a consulta das regras `CHECK` . - [] Criar `database/03_seed.sql` com a primeira parte (as categorias), rodar no **SQL Editor** e conferir as 8 categorias. - [] Se aparecer `relation "perfis" already exists` , não rodar a Parte 1 de novo; se aparecer `relation "public.perfis" does not exist` , rodar a Parte 1 antes da Parte 2. - [] Rodar `git add .` , `git commit -m "..."` e `git push` (sem nenhuma senha ou chave nos arquivos). - **Etiqueta sugerida:** Supabase (Banco de Dados) · Essencial - **Depende de:** D9-A25 - **Aula e requisitos:** docs/guia-aulas/dia09-aula26-criar-as-tabelas-com-sql.md · RN-01, RN-02, RN-11, RN-12

Data: 19/10/2026 - segunda-feira - **Cartão (Título no Trello):** D9-A27 – Praticar SELECT, INSERT e JOIN, criar a loja de teste e carregar a Loja Exemplo com 3 produtos - **User Story:** Como aluna desenvolvedora, Quero consultar o banco com SQL e carregar uma loja e 3 produtos de exemplo, Para que o catálogo tenha dados de teste quando eu ligar o site ao banco. - **Crterios de Aceite:** - As consultas de SELECT, WHERE e ORDER BY funcionaram e a categoria **Teste** foi inserida e apagada. - `select id, nome, tipo from public.perfis;` mostra uma linha **Lojista Teste** com tipo **lojista**. - A consulta com `join` mostra 3 produtos (Calça jeans reta: 1 foto e 3 tamanhos; Camiseta bsica branca: 2 fotos e 3 tamanhos; Vestido midi floral: 3 fotos e 3 tamanhos) com o nome da **Loja Exemplo**. - Os testes de regra geram os erros esperados (`produtos_preco_check`, `lojas_dono_id_key` e "Cada produto pode ter no mximo 5 fotos.") e as fotos de teste foram apagadas. - `database/03_seed.sql` tem as duas partes, com o `v_dono` ainda com zeros no arquivo do projeto. - **Checklist interna do carto:** - [] No **SQL Editor** (em portugus: **Editor SQL**), rodar uma por vez as consultas de `select`, `where` e `order by` do Passo 1 (a terceira, com `like '%S'`, devolve Saias e Shorts). - [] Rodar o `insert` da categoria **Teste**, ver a nova linha e rodar o `delete` com `where` (nunca um `delete` sem `where`). - [] No painel, abrir **Authentication** (em portugus: **Autenticao**) > **Users** (em portugus: **Usuários**) > **Add user** (em portugus: **Adicionar usuário**) > **Create new user** (em portugus: **Criar novo usuário**). - [] Preencher **Email** (que você acesse) e **Password** (em portugus: **Senha**) com pelo menos 6 caracteres e marcar **Auto Confirm User** (em portugus: **Confirmar usuária automaticamente**), se existir. - [] Em **User Metadata** (em portugus: **Metadados do usuário**), se existir, colar `{"nome": "Lojista Teste", "tipo": "lojista"}`; clicar em **Create user** (em portugus: **Criar usuário**) e copiar o **UID**. - [] Rodar `select id, nome, tipo from public.perfis;`. Se aparecer **Sem nome** e **cliente**, apagar o perfil (`delete from public.perfis where id = '...'`) e recriar como a aula indica. - [] Abrir `docs/guia-aulas/materiais/03_seed_parte2.sql`, selecionar tudo (Ctrl+A; Mac: Cmd+A), copiar e colar no final de `database/03_seed.sql`. - [] Copiar do `03_seed.sql` só o bloco do `do $$` até o `$$;`, colar em uma nova consulta no painel e trocar os zeros de `v_dono` pelo UID da loja de teste (somente na cópia do painel, não no arquivo do projeto). - [] Clicar em **Run** (em portugus: **Executar**) e conferir **Success**. - [] Rodar a consulta das lojas e a consulta com `join` e conferir a tabela do Passo 5. - [] Rodar os testes das regras do banco (preço zero e segunda loja da mesma loja) e o teste do limite de 5 fotos; limpar as fotos de teste com `delete ... where url like 'https://exemplo.com/%'`. - [] Anotar no quadro do Trello que a RLS fica **desligada** até o Dia 13 e que só dados de teste são usados. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Supabase (Banco de Dados) · Essencial - **Depende de:** D9-A26 - **Aula e requisitos:** docs/guia-aulas/dia09-aula27-sql-basico-select-insert-e-join.md · não se aplica (preparação dos dados; base de RF-01, RF-06 e RF-16; reforça RN-02 e RN-12)

Dia 10 · 20/10/2026 · terça-feira

Entrega do dia: catálogo lendo do banco (produtos, categorias e lojas do Supabase), com filtros feitos na consulta e paginação de 12 produtos.

Observações do dia: - Continuam valendo as regras do plano para os Dias 9 a 12: RLS **desativada**, lojista de teste e nenhum login ainda. No **Table Editor** (em português: **Editor de tabelas**) cada tabela deve mostrar **RLS disabled**. - A Aula 29 mistura painel do Supabase, arquivos novos e seis trocas no `catalogo.js`. Se o tempo estourar, a equipe termina o Passo 7 (apagar `dados-de-exemplo` e testar com a configuração real) antes da Aula 30; o Passo 4 (arquivos de serviço) nunca é pulado. - Na Aula 29, só a chave pública `anon` entra no código; a chave `service_role` (ou `secret`) nunca aparece em nenhum arquivo nem em mensagens de pedido de ajuda. - Na Aula 30, o teste de paginação troca `PRODUTOS_POR_PAGINA` para 2 só de forma **temporária**; o valor volta para 12 no fim.

Data: 20/10/2026 - terça-feira - **Cartão (Título no Trello):** D10-A28 – Buscar os dados com `async/await` e `fetch`, com "Carregando..." e erro (arquivos JSON de teste) - **User Story:** Como cliente, Quero ver "Carregando..." enquanto os dados chegam e uma mensagem clara quando falham, Para que eu saiba o que está acontecendo e nunca veja uma tela em branco. - **Críticos de Aceite:** - O catálogo mostra os 6 produtos vindos dos arquivos JSON. - Com **Slow 3G** (em português: **3G lento**) na aba **Network** (em português: **Rede**), o aviso "Carregando..." fica visível por mais tempo. - Com o nome do arquivo errado aparece "Não foi possível carregar os dados de exemplo." - Com **Offline** aparece uma mensagem em português, nunca uma tela em branco (RF-21). - **Checklist interna do cartão:** - [] Criar a pasta provisória `dados-de-exemplo` e o arquivo `dados-de-exemplo/lojas.json` com o conteúdo da aula. - [] Abrir `docs/guia-aulas/materiais/dados-de-exemplo-produtos.json`, selecionar tudo (Ctrl+A; Mac: Cmd+A), copiar e colar em um arquivo novo `dados-de-exemplo/produtos.json`. - [] No `catalogo.js`, importar o `ErroApp`, apagar a constante `produtosFicticios` e apagar a função `carregarLojasFicticias`. - [] Colar a função `carregarLojasDoArquivo` (usa `fetch` e `Promise.all`) antes de `iniciar` e trocar `iniciar` pela versão `async`. - [] Abrir o catálogo pelo Live Server e conferir que funciona como antes, agora com dados dos arquivos. - [] Abrir F12 > **Network** (em português: **Rede**), escolher **Slow 3G** (em português: **3G lento**) onde diz **No throttling** (em português: **Sem limitação**) e recarregar (F5) para ver o "Carregando..."; depois voltar para **No throttling**. - [] Provocar o erro 404: trocar `lojas.json` por `loja.json` no código, ver a caixa vermelha e desfazer. - [] Provocar falha de conexão: marcar **Offline** na aba **Network**, recarregar, ver a mensagem em português e voltar para **No throttling**. - [] Se der `await is only valid in async functions`, conferir que a função é `async function`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta**

sugerida: JavaScript · Essencial - **Depende de:** D9·A27 - **Aula e requisitos:** docs/guia-aulas/dia10-aula28-javascript-assincrono.md · RF-21

Data: 20/10/2026 - terça-feira - **Cartão (Título no Trello):** D10·A29 – Conectar o site ao Supabase com supabase-js e listar os produtos do banco - **User Story:** Como cliente, Quero que o catálogo mostre os produtos que estão no banco, Para que o que a loja cadastrou apareça de verdade no site. - **Critérios de Aceite:** - Com os valores de exemplo em `js/config.js`, o catálogo mostra a caixa vermelha "O sistema ainda não está conectado ao banco de dados. Preencha a URL e a chave do Supabase no arquivo js/config.js." - Com os valores certos, o catálogo mostra os 3 produtos do banco, as 8 categorias como chips e a Loja Exemplo na lista de lojas, e os filtros continuam funcionando. - Procurando `service_role` e `secret` com Ctrl+Shift+F (Mac: Cmd+Shift+F) há 0 resultados no código; só a chave pública `anon` aparece. - A pasta `dados-de-exemplo` não existe mais. - Trocar um preço no **Table Editor** do Supabase e recarregar o catálogo mostra o novo preço. - **Checklist interna do cartão:** - [] No painel do Supabase, abrir **Project Settings** (em português: **Configurações do projeto**) > **API** (ou o botão **Connect**, em português: **Conectar**) e copiar o **Project URL** (em português: **URL do projeto**). - [] Copiar a chave `anon` / `public` (em projetos novos, **publishable key**, em português: **chave publicável**). Não copiar a chave `service_role` (ou `secret`). - [] Criar `js/config.js` com o conteúdo da aula e trocar os dois valores de exemplo pelos do seu projeto, mantendo as aspas. - [] Criar `js/supabaseClient.js` (carrega a biblioteca por CDN e cria o cliente). - [] Criar `js/servicos/categoriaServico.js` e `js/servicos/lojaServico.js` (cada função devolve dados ou lança `ErroApp`, dentro de `try/catch`). - [] No `produtoServico.js`, trocar o começo do arquivo pelas importações e constantes e colar a função `listarProdutosAtivos` (só produtos ativos; RN-08) antes do bloco "Painel da lojista". - [] No `Produto.js`, colar o método estático `deLinha` antes do comentário `// ----- Leitura -----`. - [] No `catalogo.js`, fazer as cinco trocas da aula, na ordem: importações; dados fictícios saem; nova `atualizarCatalogo`; chips usam `categoria.nome`; apagar `carregarLojasDoArquivo` e trocar `iniciar`. - [] Apagar `dados-de-exemplo/lojas.json` e `dados-de-exemplo/produtos.json`. - [] Teste 1 (antes de preencher a configuração): abrir o catálogo e ver o aviso da caixa vermelha. - [] Teste 2 (configuração preenchida): conferir os 3 produtos, as 8 categorias e a Loja Exemplo. - [] No **Table Editor**, conferir que cada tabela mostra **RLS disabled**. - [] Procurar `service_role` e `secret` em todo o código: 0 resultados. - [] Rodar `git add .`, `git commit -m "..."` e `git push`, sem colar a chave em nenhuma mensagem de ajuda. - **Etiqueta sugerida:** Supabase (Banco de Dados), JavaScript · Essencial - **Depende de:** D10·A28 - **Aula e requisitos:** docs/guia-aulas/dia10-aula29-conectar-o-front-ao-supabase.md · RF-01, RF-21

Data: 20/10/2026 - terça-feira - **Cartão (Título no Trello):** D10·A30 – Passar os filtros para a consulta ao banco e criar o "Carregar mais produtos" (12 por página) - **User Story:** Como cliente,

Quero filtrar por tipo, loja e tamanho, buscar pelo nome e carregar mais produtos aos poucos, Para que o catálogo seja rápido e traga só o que eu pedi. - **Critérios de Aceite:** - Cada filtro (tipo, loja, tamanho e busca) funciona sozinho e combinado, e **Todas** limpa o de tipo (RF-02, RF-03, RF-04). - O filtro de tamanho **G** não mostra o produto sem estoque nesse tamanho. - Digitar **CAMISETA** acha a camiseta (RF-05); digitar **zzz** mostra "Nenhum produto encontrado." - Com **PRODUTOS_POR_PAGINA = 2**, o botão **Carregar mais produtos** aparece, traz o próximo produto sem repetir nenhum e some no fim; ao mudar o filtro, a lista volta à primeira página; depois do teste o valor volta para 12 (RF-01). - **catalogo.html?categoria=3** abre já filtrado e com o chip **Vestidos** marcado. - Sem internet (**Offline**), aparece uma mensagem em português. - **Checklist interna do cartão:** - [] No **produtoServico.js**, colar as três funções de apoio antes de **listarProdutosAtivos**: **escaparCuringas**, **montarConsultaDosProdutosAtivos** e **erroAoListarProdutos**. - [] Trocar **listarProdutosAtivos** pela versão com filtros e colar a função da paginação antes do bloco "Painel da lojista". - [] No **catalogo.html**, colar o **<p class="paginacao">** com o botão **Carregar mais produtos** logo antes de **</section>**. - [] No final de **css/paginas.css**, colar a seção do botão "Carregar mais". - [] Substituir **todo** o conteúdo de **js/paginas/catalogo.js** pelo arquivo completo da aula (filtros no banco, paginação, busca com espera e leitura do endereço). - [] Testar o filtro de tipo (**Vestidos**), de loja (**Loja Exemplo**) e de tamanho (**G**). - [] Testar a busca com **CAMISETA** e com **zzz**. - [] Testar a paginação trocando temporariamente **const PRODUTOS_POR_PAGINA = 12;** por **2**; clicar em **Carregar mais produtos** e conferir que não repete produto; **voltar o valor para 12**. - [] Abrir **catalogo.html?categoria=3** na barra de endereço e conferir o filtro já aplicado. - [] Na aba **Network** (em português: **Rede**) do F12, filtrar por **produtos** e conferir uma consulta por filtro, com parâmetros na URL. - [] Conferir **Offline** na aba **Network** e a mensagem em português. - [] Se produtos se repetirem, conferir o **.order("id")** de desempate; se a busca consultar a cada letra, conferir **setTimeout** e **clearTimeout** em **configurarFiltros**. - [] Rodar **git add .**, **git commit -m "..."** e **git push**. - **Etiqueta sugerida:** Supabase (Banco de Dados), JavaScript · Essencial - **Depende de:** D10-A29 - **Aula e requisitos:** docs/guia-aulas/dia10-aula30-catalogo-e-filtros-consultando-o-banco.md · RF-01, RF-02, RF-03, RF-04, RF-05 (Desejável), RF-21

Dia 11 · 21/10/2026 · quarta-feira

Marco: Marco 3 · CRUD com imagens conectado ao Supabase, versão 0.5 (tag **v0.5**). Evidência: sistema rodando com o banco em nuvem.

Entrega do dia: loja e produtos cadastrados, editados, desativados e excluídos pelo sistema, com até 5 fotos por produto guardadas no Supabase Storage.

Observações do dia: - **Regra para os Dias 9 a 12:** RLS **desativada**, a política provisória de Storage (`dev: envio de fotos`) existe só até o Dia 13 e as telas do painel usam a **lojista de teste** (o id provisório fica em `js/config.js`). Qualquer pessoa com o endereço do site poderia gravar: usar só dados de teste e não divulgar o link. - A Aula 33 é uma das mais cheias do curso (nove trocas no formulário, três arquivos novos e o painel do Storage). Se o tempo estourar, **o que pode sair primeiro é a reordenação das fotos** (botões "Mover para cima" e "Mover para baixo"), depois o upload de arquivos (usando o campo de endereço/URL como plano B); os passos de banco e de segurança não são pulados. - Anotar no quadro do Trello: "apagar a política provisória de fotos no Dia 13". - Se algum passo de uma aula não terminar, anotar o número do passo e continuar dele no começo da aula seguinte, sem passar à próxima aula com o cartão anterior aberto.

Data: 21/10/2026 - quarta-feira - **Cartão (Título no Trello):** D11-A31 – Cadastrar a loja e os produtos (INSERT) como lojista de teste - **User Story:** Como lojista, Quero cadastrar a minha loja e os meus produtos com categoria, preço e tamanhos com estoque, Para que as clientes vejam o que eu vendo no catálogo. - **Crterios de Aceite:** - A tela **Minha loja** carrega a loja de teste, grava a edição ("Loja salva com sucesso.") e mostra o WhatsApp só com dígitos começando por `55` (por exemplo, `5527999998888`) (RF-15, RN-10). - O produto **Short jeans** é cadastrado e aparece no catálogo; filtrando por tamanho **G** (estoque 0), ele não aparece (RF-16). - Tamanho repetido ("Este tamanho já foi adicionado.") e preço `0` ("O preço deve ser maior que zero.") mostram erro ao lado do campo (RN-02). - Cadastrar um produto com o nome `<img src=x onerror=alert(1)>` mostra o texto como texto no catálogo, sem alerta. - No **Table Editor** (em português: **Editor de tabelas**), `lojas`, `produtos` e `tamanhos` têm as linhas novas; uma segunda loja da mesma lojista é recusada (RN-01). - **Checklist interna do cartão:** - [] No painel do Supabase, abrir **Authentication** (em português: **Autenticação**) > **Users** (em português: **Usuários**) e copiar o **UID** da lojista de teste. - [] No final de `js/config.js`, colar a constante provisória `LOJISTA_DE_TESTE_ID` e trocar o valor pelo UID copiado. - [] Criar `js/servicos/errosSupabase.js`, que traduz os erros do Supabase para `ErroApp` em português. - [] No `lojaServico.js`, trocar o começo do arquivo pelas novas importações e colar no final o bloco "Painel da lojista" (validação, `obterMinhaLoja` e `salvarLoja`). - [] No `painel-loja.html`: trocar o `<header>` por `<header class="cabecalho" id="cabecalho">` `</header>`, acrescentar `hidden` na linha do `<form ... id="formulario-loja" novalidate hidden>` e colar `<script type="module" src="js/paginas/painelLoja.js"></script>` antes de `</body>`. - [] Criar `js/paginas/painelLoja.js`. - [] Testar a tela Minha loja: editar a descrição, salvar e recarregar; digitar o WhatsApp `27 99999-8888` e ver `5527999998888`; apagar o nome e ver "Informe o nome da loja."; digitar um link de mapa sem `https://` e ver o erro do link. - [] Conferir no **Table Editor** > `lojas` que o WhatsApp está só com dígitos. - [] No `produtoServico.js`, trocar a importação do `ErroApp` por ela mais `Produto` e a tradução de erros; colar no final `montarLinhas`, `conferirComOModelo` e `criarProduto`. - [] Substituir **todo** o conteúdo de `painel-`

`produto-form.html` e de `js/paginas/painelProdutoForm.js` pelas versões da aula (linhas de tamanho criadas pelo JavaScript). - [] Testar o formulário: clicar em **Adicionar tamanho** duas vezes e em **Remover tamanho** em uma; cadastrar `Short jeans`, categoria **Shorts**, preço `79,90`, tamanhos `M` (estoque 3) e `G` (estoque 0); conferir que o produto aparece no catálogo e não aparece com o filtro **G**. - [] Testar erros: tamanho `M` repetido e preço `0`. - [] Teste de segurança: cadastrar um produto com o nome `<img src=x onerror=alert(1)>`, conferir que aparece como texto e apagá-lo depois no **Table Editor**. - [] Rodar `git add .`, `git commit -m "..."` e `git push` (o id provisório não é senha, mas será apagado no Dia 13). - **Etiqueta sugerida:** Supabase (Banco de Dados), JavaScript · Essencial - **Depende de:** D10-A30 - **Aula e requisitos:** docs/guia-aulas/dia11-aula31-cadastro-de-lojas-e-produtos.md · RF-15, RF-16, RN-01, RN-02, RN-10

Data: 21/10/2026 - quarta-feira - **Cartão (Título no Trello):** D11-A32 – Editar, desativar, ativar e excluir produtos (UPDATE e DELETE) - **User Story:** Como lojista, Quero corrigir, desativar e excluir os meus produtos, Para que o catálogo mostre só o que eu realmente vendo. - **Critérios de Aceite:** - A tabela **Meus produtos** lista os produtos da loja, ativos e inativos, e vira blocos em 360 px. - Editar abre o formulário com os dados preenchidos, salva ("Produto atualizado com sucesso.") e a mudança aparece no catálogo (RF-17). - Desativar tira o produto do catálogo ("Produto desativado: ele não aparece mais no catálogo."); Ativar traz de volta (RN-08). - Excluir um produto novo funciona, com confirmação; excluir um produto já pedido mostra "Este produto já foi pedido; desative-o em vez de excluir." (RF-18). - `painel-produto-form.html?id=abc` mostra "Produto não encontrado." - **Checklist interna do cartão:** - [] No `produtoServico.js`, colar a constante `MENSAGEM_PRODUTO_JA_PEDIDO` antes de `normalizarTamanho` e as funções `listarProdutosDaLoja` e `obterProdutoParaEdicao` antes de `montarLinhas`. - [] Colar no final do arquivo `atualizarProduto`, `definirAtivo` e `excluirProduto`, nessa ordem. - [] Substituir **todo** o conteúdo de `painel-produtos.html` e criar `js/paginas/painelProdutos.js`. - [] No `painelProdutoForm.js`, fazer as cinco trocas: cabeçalho e importações; `idNaUrl` com `URLSearchParams` (modo edição); salvar criando **ou** atualizando; colar `preencherFormulario` antes de `iniciar`; trocar `iniciar`. - [] Testar a lista (Ativo/Inativo, 360 px em blocos). - [] Testar **Editar** no Short jeans (preço `69,90`, tirar o tamanho G) e conferir o novo preço no catálogo. - [] Testar **Desativar** (some do catálogo) e **Ativar** (volta). - [] Testar **Excluir** com a confirmação do navegador. - [] No **SQL Editor** (em português: **Editor SQL**), gravar um pedido de teste (`insert into public.pedidos ...` com os ids da sua conta), tentar excluir a **Camiseta básica branca** na tela e ver a mensagem de "desative-o"; depois apagar o pedido de teste com `delete from public.pedidos where id = '11111111-1111-1111-1111-111111111111'`; - [] Abrir `painel-produto-form.html?id=abc` e ver "Produto não encontrado." - [] Se o formulário abrir vazio ao editar, conferir o `?id=` do link **Editar** e a chamada de `preencherFormulario` em `iniciar`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Supabase (Banco de Dados),

JavaScript · Essencial - **Depende de:** D11-A31 - **Aula e requisitos:** docs/guia-aulas/dia11-aula32-
editar-e-excluir-produtos.md · RF-17, RF-18, RN-08, RF-21

Data: 21/10/2026 - quarta-feira - **Cartão (Título no Trello):** D11-A33 – Enviar até 5 fotos por produto ao Supabase Storage, com capa, reordenação e versão 0.5 (Marco 3) - **User Story:** Como lojista, Quero cadastrar até 5 fotos por produto e escolher qual é a capa, Para que as clientes vejam a peça de vários ângulos. - **Critérios de Aceite:** - Um produto com 3 fotos mostra a primeira como capa no catálogo (selo **Capa**) e todas na edição; os botões **Mover para cima/baixo** mudam a capa (RF-19, RN-12, CT-10). - No Storage, os arquivos ficam em `loja/produto/uuid.jpg` e uma foto de câmera com mais de 1 MB fica com cerca de 200 KB (CT-25). - Escolher a 6ª foto, um arquivo `.gif` ou um arquivo maior que 2 MB é recusado com mensagem clara, e as fotos já escolhidas continuam na lista (CT-16). - Excluir o produto apaga os arquivos do Storage (RF-18). - Sem foto, o card mostra a imagem padrão "Sem foto" (RN-12). - A tag `v0.5` aparece no GitHub (**Marco 3**). - **Checklist interna do cartão:** - [] No painel do Supabase, abrir **Storage** (em português: **Armazenamento**) e clicar em **New bucket** (em português: **Novo bucket**); em **Name** (em português: **Nome**) escrever `produtos`, ligar **Public bucket** (em português: **Bucket público**) e clicar em **Create** (em português: **Criar**). - [] No **SQL Editor** (em português: **Editor SQL**), rodar a política provisória `create policy "dev: envio de fotos" on storage.objects for insert to anon ...` (só permite enviar) e anotar no quadro que ela deve ser apagada no Dia 13. - [] Criar `js/servicos/storageServico.js`, `js/ui/imagem.js` (reduz a foto no navegador para cerca de 200 KB) e `js/ui/listaDeFotos.js` (selo **Capa**, **Mover para cima**, **Mover para baixo** e **Remover**). - [] No `produtoServico.js`: trocar as importações (`MAXIMO_DE_FOTOS` e `Storage`); trocar `validarProduto` pela versão com a regra das 5 fotos; colar `enviarFotosNovas` antes de `criarProduto`; trocar `criarProduto`, `atualizarProduto` e `excluirProduto` pelas versões com fotos. - [] No `painel-produto-form.html`, colar a seção `<section aria-labelledby="titulo-fotos">` antes de `<div class="acoes-formulario">`. - [] No `painelProdutoForm.js`, fazer as nove trocas na ordem da aula (importações, campos das fotos, lista de fotos, leitura do formulário com `fotos.obterParaSalvar()`, erro ao lado do campo, eventos das fotos, mensagem de envio, campo de endereço da imagem como alternativa, `preencherFormulario`). - [] Testar (CT-10): cadastrar um produto com 3 fotos, usar **Mover para baixo** na primeira e salvar; conferir a capa no catálogo e as 3 fotos na edição. - [] Conferir no **Storage** > bucket `produtos` a pasta com o id da loja e a pasta do produto. - [] Testar (CT-25): enviar uma foto de câmera com mais de 1 MB e menos de 2 MB e conferir o arquivo com cerca de 200 KB (JPG). - [] Testar (CT-16): escolher 6 fotos, um `.gif` e um arquivo maior que 2 MB; conferir a mensagem e que as fotos da lista não somem. - [] Excluir o produto de teste em **Meus produtos** e conferir que os arquivos somem do Storage. - [] Plano B: se o envio falhar, informar o endereço (URL) de uma imagem da internet no campo que aparece e salvar. - [] Se der `new row violates row-level security policy`, conferir o nome do bucket e a

política; se as fotos não aparecerem no catálogo, conferir **Public bucket**. - [] Rodar `git add .`, `git commit -m "..."` e `git push`; criar e enviar a tag `v0.5` (`git tag v0.5` e `git push origin v0.5`). - **Etiqueta sugerida:** Supabase (Banco de Dados), Segurança · Importante - **Depende de:** D11-A32 - **Aula e requisitos:** docs/guia-aulas/dia11-aula33-fotos-no-supabase-storage.md · RF-19, RN-12, RF-18 (apagar as fotos ao excluir); CT-10, CT-16 e CT-25 · **Marco 3**

Dia 12 · 22/10/2026 · quinta-feira

Entrega do dia: páginas de produto e de loja com dados do banco, sacola agrupada por loja (localStorage) e "Finalizar sacola" montando um pedido por loja com botão de WhatsApp por loja. **A gravação dos pedidos no banco só entra no Dia 13**, depois do login (RF-10 fica completo no D13-A39).

Observações do dia: - **Regra para os Dias 9 a 12:** RLS **desativada**, política provisória de Storage ainda existente e lojista de teste (nenhum login). Nada é gravado nas tabelas `pedidos` e `itens_pedido` hoje: elas devem continuar **vazias** (confira no **Table Editor**, em português: **Editor de tabelas**). - Hoje a tela "Pedidos enviados" mostra o texto "Pedidos gravados!" mesmo sem gravar; isso é esperado e vira verdade só no Dia 13. Registrar isso no cartão do Trello para a equipe não achar que é erro. - É o dia mais cheio do módulo de pedidos (três aulas longas). Se o tempo apertar, **o primeiro item a sair é o link "Ver no mapa" da página da loja (RF-07)**; a galeria acessível por teclado, a sacola com duas lojas e o botão de WhatsApp por loja ficam. - Para testar a sacola é preciso ter **duas lojas** com produtos ativos e WhatsApp cadastrado (a Loja Exemplo e uma segunda loja cadastrada em **Minha loja** na Aula 31 ou criada hoje, com outra lojista de teste). - Cada aula termina com `git commit` e `git push`; só passar ao cartão seguinte quando o anterior estiver em "Concluído".

Data: 22/10/2026 - quinta-feira - **Cartão (Título no Trello):** D12-A34 – Montar as páginas de produto (galeria) e de loja (endereço e mapa) usando o id na URL - **User Story:** Como cliente, Quero ver a página do produto com fotos, tamanhos e quantidade e a página da loja com endereço e WhatsApp, Para que eu escolha a peça certa e saiba como falar com a loja. - **Critérios de Aceite:** - A página do produto mostra galeria com miniaturas, preço, "Vendido por Loja Exemplo", descrição e tamanhos; o tamanho **G** (estoque 0) aparece riscado e não pode ser escolhido (RF-08). - As miniaturas trocam a foto grande com o mouse e com o teclado (Tab + Enter); o título da aba passa a ser o nome do produto. - Sem tamanho escolhido aparece "Escolha um tamanho." e com quantidade `0` aparece "A quantidade mínima é 1." ao lado do campo (RN-02). - `produto.html?id=abc`, `produto.html` sem id e `loja.html?id=abc` mostram "Produto não encontrado." ou "Loja não encontrada.", sem tela em branco; um produto inativo também mostra

"Produto não encontrado." (RN-08). - A página da loja mostra nome, descrição, endereço, o botão **Conversar no WhatsApp** com link `https://wa.me/5527999999999?text=...` (RN-10), os produtos e o botão **Ver no mapa** só quando existe link (RF-06, RF-07). - **Checklist interna do cartão:** - [] Abrir um card do catálogo e olhar a barra de endereço: o `?id=...` é o código do produto. - [] No `produtoServico.js`, colar a função `obterProduto` antes do bloco "Painel da lojista". - [] No `lojaServico.js`, colar a seção "Páginas públicas" antes do comentário `// ----- Painel da lojista -----`. - [] No final de `js/ui/formatadores.js`, acrescentar `criarLinkDoWhatsapp` (monta `https://wa.me/<número>?text=<mensagem codificada>` com `encodeURIComponent`). - [] Substituir **todo** o conteúdo de `produto.html` e criar `js/paginas/produto.js`. - [] Substituir **todo** o conteúdo de `loja.html` e criar `js/paginas/loja.js`. - [] Testar o **Vestido midi floral**: foto grande, 3 miniaturas, **Vendido por Loja Exemplo**, tamanho **G** riscado. - [] Testar a galeria com o mouse e com Tab + Enter. - [] Clicar em **Adicionar à sacola** sem tamanho, depois com quantidade `0`, depois com `2` (por enquanto aparece só a mensagem de teste). - [] Abrir `produto.html?id=abc`, `produto.html` e `loja.html?id=abc` e conferir o link **Voltar ao catálogo**. - [] Desativar um produto em **Meus produtos** e abrir o endereço dele. - [] Na página da loja, clicar em **Conversar no WhatsApp** (abre em nova aba) e conferir o endereço `https://wa.me/...`. - [] Em **Minha loja**, apagar o link do mapa, salvar e conferir que o botão **Ver no mapa** some da página pública (RF-07). - [] Confirmar que a página usa só `textContent` e `createElement` (nenhum `innerHTML` em `js/`). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript, Supabase (Banco de Dados) · Essencial - **Depende de:** D11-A33 - **Aula e requisitos:** docs/guia-aulas/dia12-aula34-paginas-de-produto-e-de-loja.md · RF-06, RF-07, RF-08, RN-02, RN-08, RN-10

Data: 22/10/2026 - quinta-feira - **Cartão (Título no Trello):** D12-A35 – Criar a sacola com localStorage, agrupada por loja, com subtotais, total e contador - **User Story:** Como cliente, Quero juntar peças de lojas diferentes na mesma sacola e ver o subtotal de cada loja e o total, Para que eu compre de várias lojas de uma vez sem perder o que escolhi ao recarregar a página. - **Crêterios de Aceite:** - Peças de duas lojas ficam na mesma sacola, agrupadas por loja, cada bloco com o subtotal da loja e, no fim, o **Total**; o botão diz "Finalizar sacola (2 pedidos)" (RF-09, RN-03, CT-04). - Recarregar a página (F5) mantém itens, subtotais e total (CT-05). - O contador no cabeçalho soma ao adicionar, diminui ao remover e some em zero. - Quantidade `0` mostra "A quantidade mínima é 1." e volta ao valor anterior (RN-02); o mesmo produto no mesmo tamanho vira uma linha só com quantidade `2`. - Sem itens aparece "Sua sacola está vazia. Ver o catálogo."; na aba **Application** (em português: **Aplicativo**) existe a chave `sacola` no **Local Storage** (em português: **Armazenamento local**). - **Checklist interna do cartão:** - [] Criar `js/modelos/Sacola.js` (adicionar, mudar a quantidade, remover, salvar no `localStorage`; aceita lojas diferentes). - [] No `cabecalho.js`: trocar o comentário e a importação da `Sacola`; trocar os links para o link da sacola pedir o contador (`comContador: true`); colar `atualizarContadorSacola` antes de `criarLink`;

trocar `criarLink` e `montarCabecalho` pelas versões da aula. - [] No `css/componentes.css`, colar a seção do contador antes de `/* ----- Erro ao lado de um campo de formulário ----- */`. - [] No `produto.js`: trocar o comentário do começo; importar `atualizarContadorSacola` e `Sacola`; trocar o trecho final do `submit` para gravar na sacola e mostrar "Peça adicionada à sacola. Ver sacola ou continuar comprando." - [] Substituir **todo** o conteúdo de `sacola.html` e criar `js/paginas/sacola.js`. - [] Testar (CT-04): adicionar uma peça da **Loja Exemplo** e uma de outra loja e conferir que o site não pergunta nada ao misturar. - [] Abrir **Sacola** e conferir os blocos por loja, subtotais, **Total** e "Finalizar sacola (2 pedidos)". - [] Mudar a quantidade para `3` (valores atualizam sem recarregar) e para `0` (erro e volta ao valor anterior). - [] Clicar em **Remover**, conferir o contador e o foco no próximo botão; remover todos e ver "Sua sacola está vazia." - [] Adicionar duas vezes o mesmo produto e tamanho e conferir uma linha com quantidade `2`. - [] Testar (CT-05): recarregar com itens na sacola e abrir outra página; contador e valores continuam. - [] No F12 > **Application** (em português: **Aplicativo**) > **Local Storage** (em português: **Armazenamento local**), conferir a chave `sacola`. - [] Se a sacola esvaziar ao recarregar, conferir que toda alteração termina com `sacola.salvar()`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** JavaScript, Fluxo de Pedido · Essencial - **Depende de:** D12·A34 - **Aula e requisitos:** docs/guia-aulas/dia12-aula35-sacola-agrupada-por-loja.md · RF-09, RN-02, RN-03; CT-04 e CT-05

Data: 22/10/2026 - quinta-feira - **Cartão (Título no Trello):** D12·A36 – Finalizar a sacola em um pedido por loja, com o mesmo grupo_id, e botão de WhatsApp por loja - **User Story:** Como cliente, Quero finalizar a sacola e receber um pedido por loja com um botão de WhatsApp para cada uma, Para que eu combine a compra direto com cada lojista, vendo só as peças daquela loja. - **Critérios de Aceite:** - Com peças de duas lojas, **Finalizar sacola (2 pedidos)** mostra a tela "Pedidos enviados" com dois cartões, cada um com o nome da loja, o código `#xxxxxxx`, os itens e o **Total do pedido** só daquela loja (RN-03). - Cada botão de WhatsApp abre `https://wa.me/<número da loja>?text=...` com a mensagem só dos itens daquela loja (RN-10); sem WhatsApp cadastrado aparece "Esta loja ainda não cadastrou o WhatsApp..." e o botão não aparece. - Depois de finalizar, a sacola fica vazia e o contador some. - Com o preço de uma peça alterado, o sistema **não finaliza**, avisa "O preço mudou: ... (de R\$ X para R\$ Y)...", atualiza a sacola e finaliza na segunda tentativa (RN-06); com produto desativado, avisa "Estes produtos não estão mais disponíveis...". - As tabelas `pedidos` e `itens_pedido` continuam **vazias** (a gravação entra no Dia 13; RF-10 em parte). - **Checklist interna do cartão:** - [] Criar `js/modelos/Pedido.js` (um pedido por loja, todos com o mesmo `grupo_id`). - [] No `lojaServico.js`, colar a função dos contatos das lojas antes do comentário `// ----- Painel da lojista -----`. - [] Criar `js/servicos/pedidoServico.js` (versão provisória: confere o preço atual e monta os pedidos, ainda sem gravar). - [] No `sacola.js`, fazer as quatro trocas da aula: comentário do começo;

importações (inclui `mostrarCarregando`); elementos da tela (título e confirmação); trocar o botão de teste pelo código "Finalizar" (cartões, WhatsApp, confirmação e a função `finalizar`). - [] No `css/paginas.css`, colar a seção "Meus pedidos e pedidos recebidos" antes de `/* -----`
`Painel da lojista ----- */` e a seção "Sacola: confirmação do pedido" antes de `/* -----`
`--- Botão "Carregar mais" do catálogo ----- */`. - [] Preparar duas lojas com produtos ativos e WhatsApp cadastrado e adicionar uma peça de cada à sacola. - [] Clicar em **Finalizar sacola** e conferir "Enviando pedidos...", "Montando os pedidos...", os dois cartões e o selo **Enviado à loja**. - [] Clicar no botão de uma loja e conferir que o WhatsApp abre com a mensagem só dos itens dela. - [] Teste de preço (RN-06): editar o preço de uma peça da sacola em **Meus produtos**, voltar e finalizar; conferir o aviso, a sacola atualizada e a segunda tentativa. - [] Teste de produto desativado: desativar uma peça que está na sacola e finalizar. - [] Teste de loja sem WhatsApp: apagar o valor da coluna `whatsapp` no **Table Editor** (em português: **Editor de tabelas**) e finalizar. - [] Conferir que `pedidos` e `itens_pedido` continuam vazias. - [] Testar `criarLinkDoWhatsapp("5527999999999", "teste")` pelo Console (F12) se o texto da mensagem ficar estranho. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Fluxo de Pedido, JavaScript · Essencial - **Depende de:** D12-A35 - **Aula e requisitos:** docs/guia-aulas/dia12-aula36-finalizar-a-sacola.md · RF-10 (parte), RN-03, RN-06, RN-10

Dia 13 · 23/10/2026 · sexta-feira

Entrega do dia: login, recuperação de senha por e-mail e segurança básica (telas protegidas por perfil, RLS nas 8 tabelas, política provisória de fotos apagada e pedidos gravados somente pela função `criar_pedidos` do banco).

Observações do dia: - **Este é um dos dias mais cheios do curso** (três aulas pesadas). A ordem do plano vale: **login e RLS definitiva só neste dia**. Até aqui (Dias 9 a 12) a RLS ficou desativada, a política provisória de Storage existia e a lojista de teste fazia tudo. - **Ordem obrigatória na Aula 39:** primeiro mudar o **código** (Passos 1 a 6) e **só depois** ligar a RLS no banco (Passos 7 e 8). Se a RLS for ligada antes, o painel antigo (que usa a lojista de teste) deixa de funcionar. - Os passos de **segurança e de banco** da Aula 39 (RLS, `04_melhorias.sql`, apagar a política `dev: envio de fotos`) **nunca são pulados**; se faltar tempo, o que fica para depois da aula são os testes de acesso indevido pelo console (Passo 10), que a equipe refaz antes de seguir para o cartão do Dia 14. - Antes de começar a Aula 39 a equipe precisa de: uma conta de **cliente**, a **Lojista Teste** e uma **segunda lojista** (cadastrada com perfil **Lojista**, com loja e um produto). - Se o e-mail de recuperação de senha não chegar (Aula 38): o plano gratuito envia poucos e-mails por hora e pode enviar só para membros da organização; testar com o e-mail de quem criou o projeto ou de uma integrante convidada, e esperar alguns minutos se aparecer "Muitas tentativas em pouco tempo". -

A confirmação de e-mail fica **desligada** só durante as aulas (será revista no Dia 27). - O 23/10 foi tratado como dia normal de aula.

Data: 23/10/2026 - sexta-feira - **Cartão (Título no Trello):** D13-A37 – Criar cadastro, login e logout com Supabase Auth - **User Story:** Como cliente, Quero criar minha conta, entrar e sair do sistema mantendo a sessão, Para que eu finalize minhas compras e acompanhe meus pedidos. - **Critérios de Aceite:** - Criar uma conta de cliente leva à página inicial com "Conta criada! Redirecionando..." e o cabeçalho mostra **Meus pedidos**, "Olá, [nome]" e **Sair**; **Sair** volta a mostrar **Entrar** e **Cadastrar** (RF-12, RF-13). - Cadastrar com o mesmo e-mail mostra, ao lado do campo, "Este e-mail já está cadastrado. Entre na sua conta ou use outro e-mail." (CT-08); senha com 3 caracteres mostra "A senha deve ter pelo menos 6 caracteres." sem chamar o servidor; telefone `123` mostra erro. - A lojista entra e vai para `painel-loja.html`; a cliente vai para a página inicial; senha errada mostra "E-mail ou senha incorretos." - A sessão continua depois de recarregar (o `localStorage` tem uma chave que começa com `sb-`). - `select nome, tipo, telefone from public.perfis;` mostra o tipo escolhido no cadastro, e tentar trocar o tipo no banco devolve "O perfil da usuária não pode ser alterado." (RN-11). - **Checklist interna do cartão:** - [] No painel do Supabase, abrir **Authentication** (em português: **Autenticação**) > **Providers** (em português: **Provedores**) > **Email** (em português: **E-mail**) (em versões novas, **Sign In / Providers**) e desligar **Confirm email** (em português: **Confirmar e-mail**); clicar em **Save** (em português: **Salvar**). - [] Criar `js/servicos/authServico.js` (cadastrar, entrar, sair e `usuarioAtual`, devolvendo dados ou lançando `ErroApp` em português). - [] No `cabecalho.js`: trocar o começo do arquivo (comentários e importações); trocar `linksDaConta`; colar `criarItensDaUsuarua` e `sairDaConta` antes de `montarCabecalho`; trocar `montarCabecalho` pela versão `async`. - [] No `css/componentes.css`, colar a seção do nome e do botão **Sair** antes de `/* ----- Erro ao lado de um campo de formulário ----- */`. - [] Substituir **todo** o conteúdo de `cadastro.html` e de `login.html` e criar `js/paginas/cadastro.js` e `js/paginas/login.js`. - [] Testar o cadastro de cliente e conferir o cabeçalho; clicar em **Sair**. - [] Testar o e-mail repetido (CT-08), a senha curta e o telefone inválido. - [] No **SQL Editor** (em português: **Editor SQL**), rodar `select nome, tipo, telefone from public.perfis;`; cadastrar também uma conta **Lojista** e conferir. - [] Entrar com a cliente (vai para a página inicial) e com a **Lojista Teste** (vai para o painel e vê **Painel** no menu); testar senha errada. - [] Recarregar a página e abrir outra para conferir a sessão; olhar F12 > **Application** (em português: **Aplicativo**) > **Local Storage** (em português: **Armazenamento local**) e achar a chave `sb-`. - [] Tentar `update public.perfis set tipo = 'lojista' where nome = 'SEU NOME';` e conferir a recusa do banco (RN-11). - [] Se aparecer `Database error saving new user`, conferir se o `01_schema.sql` (Parte 2) foi rodado até o fim. - [] Rodar `git add .`, `git commit -m "..."` e `git push` (sem colar senhas nem chaves). - **Etiqueta**

sugerida: Segurança, Supabase (Banco de Dados) · Essencial - **Depende de:** D12-A36 - **Aula e requisitos:** docs/guia-aulas/dia13-aula37-cadastro-e-login.md · RF-12, RF-13, RN-11; CT-08

Data: 23/10/2026 - sexta-feira - **Cartão (Título no Trello):** D13-A38 – Fazer a recuperação de senha por e-mail ("Esqueci minha senha") - **User Story:** Como cliente, Quero criar uma senha nova por um link enviado ao meu e-mail, Para que eu volte a entrar na minha conta se esquecer a senha. - **Critérios de Aceite:** - E-mail inválido (`abc`) mostra "Informe um e-mail válido" ao lado do campo; um e-mail com conta e outro **sem** conta mostram a **mesma** mensagem: "Se existir uma conta com este e-mail, enviamos o link para criar uma senha nova. Confira também a caixa de spam." (RF-24). - O link do e-mail abre `recuperar-senha.html` na etapa **Senha nova e Repita a senha nova**; senhas diferentes mostram "As senhas não são iguais." e menos de 6 caracteres mostram "A senha deve ter pelo menos 6 caracteres." - Com senha válida aparece "Senha alterada! Você já está logada. Redirecionando..."; a senha nova vale e a antiga mostra "E-mail ou senha incorretos." (CT-23). - Abrir o mesmo link de novo mostra "Este link não vale mais: ele expirou ou já foi usado. Peça um novo link abaixo." - **Checklist interna do cartão:** - [] No painel do Supabase, abrir **Authentication** (em português: **Autenticação**) > **URL Configuration** (em português: **Configuração de URL**). - [] Em **Site URL** (em português: **URL do site**), escrever `http://127.0.0.1:5500` (será trocada pelo endereço publicado no Dia 15). - [] Em **Redirect URLs** (em português: **URLs de redirecionamento**), clicar em **Add URL** (em português: **Adicionar URL**) e acrescentar `http://127.0.0.1:5500/recuperar-senha.html` e `http://localhost:5500/recuperar-senha.html`; clicar em **Save** (em português: **Salvar**). - [] No `authServico.js`, colar as duas validações antes de `// ----- Cadastro, login e logout -----` e as três funções de recuperação no final do arquivo. - [] Criar `recuperar-senha.html` e `js/paginas/recuperarSenha.js` (duas etapas na mesma página). - [] Em **Entrar**, clicar em **Esqueci minha senha** e testar o e-mail inválido. - [] Enviar o link para um e-mail com conta e para um sem conta e conferir que a mensagem é a mesma. - [] Abrir o e-mail recebido (assunto parecido com **Reset Your Password**, em português: **Redefinir sua senha**), clicar no link e conferir a etapa da senha nova. - [] Testar senhas diferentes e senha curta; depois salvar uma senha nova válida e conferir que a pessoa fica logada. - [] Sair e testar a senha antiga (deve falhar) e a nova (deve funcionar) (CT-23). - [] Clicar de novo no mesmo link do e-mail e conferir a mensagem "Este link não vale mais...". - [] Se o e-mail não chegar, testar com o e-mail de um membro da organização, olhar a caixa de spam e esperar alguns minutos. - [] Se o link abrir a página inicial, conferir o endereço exato (com `.html`) em **Redirect URLs**. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Segurança, Supabase (Banco de Dados) · Importante - **Depende de:** D13-A37 - **Aula e requisitos:** docs/guia-aulas/dia13-aula38-recuperacao-de-senha.md · RF-24; CT-23

Data: 23/10/2026 - sexta-feira - **Cartão (Título no Trello):** D13-A39 – Proteger as telas por perfil, ligar a RLS e gravar os pedidos com a função criar_pedidos - **User Story:** Como cliente, Quero que só eu consiga finalizar meus pedidos e que cada lojista só mexa na própria loja, Para que meus dados e meus pedidos fiquem protegidos mesmo com a chave pública visível no navegador. -

Critérios de Aceite: - Visitante que abre `painel-loja.html` vai para `login.html?voltar=painel-loja.html`; cliente logada é levada à página inicial; a lojista entra e só vê o que é dela (RF-14, CT-09). - Sem login, "Finalizar sacola" leva ao login e, depois de entrar, **voltar à sacola** com os mesmos itens (CT-06). - Logada como cliente, finalizar uma sacola com duas lojas grava **dois pedidos** com status `novo` e o **mesmo** `grupo_id`, com preço copiado nos itens, e mostra um botão de WhatsApp por loja (RF-10, RN-03, RN-04, RN-06, CT-07). - As 8 tabelas mostram `rowsecurity = true` (24 regras em `public`: categorias 1, itens_pedido 1, lojas 4, pedidos 2, perfis 3, produto_fotos 4, produtos 5, tamanhos 4); a política `dev: envio de fotos` **não existe mais** (consulta vazia) (RN-09). - Pelo console, a lojista de outra loja recebe `data: []` ao tentar alterar o produto alheio e erro de RLS no Storage (CT-11); o insert direto em `pedidos` é recusado por **row-level security** e o preço forjado é rejeitado com "o preço mudou" (CT-21, RN-05). - A página `privacidade.html` abre, com o e-mail de contato da equipe. - **Checklist interna do cartão:** - [] Conferir que existem: uma conta de **cliente**, a **Lojista Teste** e uma **segunda lojista** (com loja e um produto). - [] Criar `js/ui/protecao.js` (`exigirLogin`, `exigirPerfil` e `enderecoDeRetornoSeguro`). - [] No `login.js` e no `cadastro.js`, fazer as trocas da aula para voltar à página de origem (`voltar` na URL). - [] Nos três HTML do painel (`painel-loja.html`, `painel-produtos.html` e `painel-produto-form.html`), trocar `<main id="conteudo" class="container">` por `<main id="conteudo" class="container" hidden>`. - [] Nos três JS do painel, importar `exigirPerfil` e trocar a função `iniciar` pela versão da aula. - [] No `lojaServico.js`, trocar as importações e as funções `obterMinhaLoja` e `salvarLoja` para usar a lojista logada. - [] No `js/config.js`, **apagar** `LOJISTA_DE_TESTE_ID` e acrescentar `EMAIL_DE_CONTATO` (trocar `EMAIL-DA-EQUIPE@exemplo.com` pelo e-mail real da equipe). - [] No `pedidoServico.js`, trocar o começo do arquivo e a função `criarPedidos` para chamar a função do banco; no `sacola.js`, trocar o começo e a função `finalizar`. - [] Criar `privacidade.html` e `js/paginas/privacidade.js`; colar a seção "Privacidade" no `css/paginas.css`, antes de `/* ----- Tablet ----- */`. - [] **Só depois de terminar o código:** criar `database/02_rls.sql`, abrir **SQL Editor** (em português: **Editor SQL**) > nova consulta, colar o arquivo inteiro e clicar em **Run** (em português: **Executar**). - [] Rodar as três consultas de conferência (8 tabelas com `true`; 24 regras; política provisória inexistente). - [] Criar `database/04_melhorias.sql` e rodar o arquivo **inteiro** depois do 02; conferir as duas funções novas. - [] Testar como visitante, como cliente e como lojista (CT-09). - [] Testar CT-06: montar a sacola sem login, finalizar, entrar e voltar à sacola. - [] Testar CT-07: finalizar logada e rodar as duas consultas (`pedidos` e `itens_pedido`) para conferir status `novo`, mesmo `grupo_id` e preços copiados. - [] Testar CT-11 no Console (F12 >

Console) como a segunda lojista, trocando `ID-DO-PRODUTO-DA-PRIMEIRA` e `ID-DA-LOJA` pelos ids reais. - [] Testar CT-21 no Console como cliente (insert direto e preço forjado). - [] Rodar `git add .`, `git commit -m "..."` e `git push`, confirmando que nenhum arquivo tem `service_role` ou senhas. - **Etiqueta sugerida:** Segurança, Fluxo de Pedido · Essencial - **Depende de:** D13-A38 - **Aula e requisitos:** docs/guia-aulas/dia13-aula39-controle-de-acesso-rls-e-criar-pedidos.md · RF-10, RF-14, RN-03, RN-04, RN-05, RN-06, RN-09; CT-06, CT-07, CT-09, CT-11, CT-21

Dia 14 · 26/10/2026 · segunda-feira

Entrega do dia: página inicial de vitrine (hero, carrossel, tipos de roupa, destaques e lojas) e fluxo completo testado e corrigido, com os 25 casos de teste registrados e o Git organizado em branches e pull requests.

Observações do dia: - A Aula 40 é muito cheia (HTML, CSS e JavaScript). Os blocos da vitrine são independentes: se o tempo apertar, **o carrossel é o primeiro item a sair** (o hero e os destaques ficam); a equipe só deixa o carrossel para depois se já tiver marcado isso no cartão. - Na Aula 41, os casos CT-17, CT-18, CT-20 e CT-24 dependem de telas do Dia 26 e ficam como **N/A** por enquanto; o CT-22 (mais de 12 produtos) pode ficar N/A e voltar no Dia 27 se faltarem produtos de teste. - Cada defeito encontrado na Aula 41 vira um cartão no quadro (modelo "Defeito: [tela] [o que aconteceu]", com caso de teste, tela e passo, o que aconteceu, o que era esperado e gravidade). Esses cartões de defeito são o trabalho da Aula 42; **não** substituem os cartões deste backlog. - Na Aula 42, o Passo 6 (resolver um conflito de propósito, em dupla) é opcional e o primeiro a sair se faltar tempo. - Ao redigir o `docs/TESTES.md`, a equipe usa só os roteiros da própria aula; o restante do tempo vai para a correção dos defeitos críticos e importantes.

Data: 26/10/2026 - segunda-feira - **Cartão (Título no Trello):** D14-A40 – Construir a página inicial de vitrine com hero, carrossel, tipos de roupa, destaques e lojas - **User Story:** Como cliente, Quero abrir o site e ver uma vitrine com peças em destaque, tipos de roupa e lojas parceiras, Para que eu decida rápido o que olhar e chegue ao catálogo já filtrado. - **Critérios de Aceite:** - A página inicial (sem login) mostra hero com título e busca, carrossel com as peças recentes, tipos de roupa, produtos em destaque (os 8 ativos mais recentes com foto e estoque), lojas parceiras, "como funciona" e chamada para lojistas (RF-23). - O carrossel troca a cada 6 segundos e também por setas < e >, pontos, teclado (← e → com o foco nele) e arraste no celular; para com o mouse ou o foco em cima, tem botão de pausa `II/▶` (CT-19). - Com "reduzir movimento" ligado no sistema operacional, o carrossel não troca sozinho e o botão de pausa não aparece. - Clicar em **Vestidos** abre o catálogo já filtrado; clicar no nome de uma loja abre a página dela; digitar `vestido` no hero e clicar em **Buscar** abre o catálogo com a busca preenchida. - Bloqueando o pedido de `lojas` na

aba **Network** (em português: **Rede**), aparece mensagem de erro, mas o carrossel e os destaques continuam (RF-21); sem produtos com foto e estoque, o carrossel some e os destaques mostram "Ainda não há peças em destaque. Volte em breve!" - Em 360 e 1280 px não há rolagem horizontal.

- **Checklist interna do cartão:** - [] Abrir o wireframe e o protótipo do catálogo (Aulas 4 e 5) e listar o que a vitrine precisa mostrar nos primeiros segundos. - [] No `lojaServico.js`, colar a função das lojas da vitrine antes do comentário `// ----- Páginas públicas -----`. - [] No `produtoServico.js`, colar `listarProdutosEmDestaque` antes de `obterProduto`. - [] Substituir **todo** o conteúdo do `index.html` pela vitrine da aula e conferir o `<link rel="stylesheet" href="css/inicio.css">` no `<head>`, depois de `paginas.css`. - [] Criar `css/inicio.css` por partes: começo e hero; carrossel; tipos de roupa, lojas, "como funciona" e chamada para lojistas; telas maiores (Tablet). - [] Criar `js/ui/carrossel.js` (exporta `criarCarrossel`) e `js/paginas/inicio.js`, de modo que cada bloco carregue sozinho (se um falhar, os outros continuam). - [] Abrir `index.html` sem estar logada e conferir todos os blocos. - [] Testar o carrossel: troca automática, setas, pontos, teclado (Tab até o carrossel e ←/→), pausa com o mouse e botão `II/▶`, arraste no modo de dispositivo do F12. - [] Ligar "reduzir movimento" no sistema operacional (Windows: **Configurações > Acessibilidade > Efeitos visuais > Efeitos de animação**; Mac: **Ajustes do Sistema > Acessibilidade > Tela > Reduzir movimento**) e conferir que o carrossel não troca sozinho. - [] Clicar em um tipo de roupa e em uma loja; testar a busca do hero com `vestido`. - [] Na aba **Network** (em português: **Rede**) do F12, clicar com o botão direito em um pedido de `lojas`, escolher **Block request URL** (em português: **Bloquear URL do pedido**), recarregar e conferir que o carrossel e os destaques continuam; desfazer o bloqueio. - [] Testar o caso sem produtos com foto e estoque. - [] Conferir em 360 e 1280 px que não há rolagem horizontal e procurar `innerHTML` em `js/` (0 resultados). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Front-end (HTML/CSS), JavaScript · Importante - **Depende de:** D13-A39 - **Aula e requisitos:** docs/guia-aulas/dia14-aula40-pagina-inicial-de-vitrine.md · RF-23, RF-21; CT-19

Data: 26/10/2026 - segunda-feira - **Cartão (Título no Trello):** D14-A41 – Depurar com as DevTools e executar os 25 casos de teste (CT-01 a CT-25) - **User Story:** Como aluna desenvolvedora, Quero testar o fluxo completo e registrar o resultado de cada caso, Para que a equipe saiba o que funciona e o que precisa ser corrigido antes de publicar. - **Critérios de Aceite:** - A equipe usou as abas **Console**, **Network** (em português: **Rede**) e **Elements** (em português: **Elementos**) e simulou **Offline**, vendo mensagem em português e nenhuma tela em branco (CT-15). - `docs/TESTES.md` existe, com a coluna **Resultado** preenchida (`OK`, `FALHOU` ou `N/A`) nos 25 casos; CT-17, CT-18, CT-20 e CT-24 estão como `N/A`. - Cada caso `FALHOU` tem explicação em **Observações** e um cartão de defeito no quadro, com gravidade (crítico, importante ou desejável). - O CT-12 foi feito: o nome `<img src=x onerror=alert(1)>` aparece como texto, sem alerta, no catálogo, no produto e

na sacola. - A tabela **Resumo** do arquivo mostra o total de **OK**, **FALHOU** e **N/A**. - **Checklist interna do cartão:** - [] Abrir o catálogo e apertar F12 (Mac: Cmd+Option+I); no **Console**, digitar **+ 1**; em **Elements** (em português: **Elementos**), clicar no ícone de seta e em um card, e mudar o preço de teste; em **Network** (em português: **Rede**), recarregar e olhar status 200 e 404, **Headers** (em português: **Cabeçalhos**) e **Response** (em português: **Resposta**). - [] Simular falha de conexão: trocar **No throttling** por **Offline**, recarregar o catálogo e trocar um filtro; voltar para **No throttling**. - [] Criar a pasta **docs** (se não existir) e o arquivo **docs/TESTES.md** com o conteúdo da aula. - [] Combinar as duplas e dividir os casos (por exemplo, CT-01 a CT-08, CT-09 a CT-16 e CT-17 a CT-25). - [] Preencher o quadro do topo do **TESTES.md** (versão testada, endereço do site, data, navegador e quem testou). - [] Em cada caso, uma pessoa executa e a outra observa e anota; trocar de papel a cada caso. - [] Marcar **N/A** nos casos CT-17, CT-18, CT-20 e CT-24 (dependem do Dia 26); para o CT-22, criar produtos de teste ou marcar **N/A**; para o CT-23, usar o e-mail de um membro da organização do Supabase. - [] Fazer o CT-12: cadastrar o produto com o nome `<img src=x onerror=alert(1)>`, olhar o catálogo, o produto e a sacola, e depois apagar o produto. - [] Fazer os CT-13 e CT-14 com **Toggle device toolbar** (em português: **Alternar barra de ferramentas do dispositivo**) em 360 e 1280 px, e o fluxo de pedido só com o teclado. - [] Fazer pelo Console os roteiros de acesso indevido do **TESTES.md** (CT-11 e CT-21), copiando o roteiro exatamente. - [] Para cada caso **FALHOU**, criar um cartão "Defeito: [tela] [o que aconteceu]" com caso de teste, tela e passo, o que aconteceu, o que era esperado e gravidade. - [] Contar **OK**, **FALHOU** e **N/A** e preencher a tabela **Resumo**. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Testes, Documentação · Essencial - **Depende de:** D14-A40 - **Aula e requisitos:** docs/guia-aulas/dia14-aula41-depuracao-e-casos-de-teste.md · verificação de RF-01 a RF-10, RF-12 a RF-16, RF-18, RF-19, RF-21, RF-23 e RF-24 (CT-01 a CT-25)

Data: 26/10/2026 - segunda-feira - **Cartão (Título no Trello):** D14-A42 – Corrigir os defeitos graves com branch, pull request revisado e merge - **User Story:** Como aluna desenvolvedora, Quero corrigir cada defeito em uma branch própria e juntar com um pull request revisado, Para que a correção entre na main sem quebrar o que já funcionava. - **Critérios de Aceite:** - Cada defeito crítico ou importante corrigido tem uma branch, um pull request aprovado por **outra** integrante e o merge na **main**; o cartão do defeito foi para "Concluído". - O caso de teste que falhou **passa** agora e o **docs/TESTES.md** foi atualizado (**FALHOU** para **OK**, com a data). - A busca por **innerHTML** em **js/** dá 0 resultados e a busca por **service_role** não encontra chave nenhuma (RF-21: nenhuma tela em branco). - `git log --oneline --graph` mostra as branches e os merges, com commits de várias integrantes. - **Checklist interna do cartão:** - [] No quadro, escolher os cartões de defeito **críticos** e **importantes**; cada integrante assume um e o move para "Em Andamento" (quem ficou sem defeito corrige um desejável ou faz a revisão de código). - [] No terminal, voltar para a **main** (`git switch main`), trazer as novidades (`git pull`) e criar a branch

(`git switch -c correcao-nome-do-defeito`); conferir com `git branch` . - [] Se não houver defeito, usar o exercício de apoio: branch `correcao-email-de-contato` e trocar `EMAIL-DA-EQUIPE@exemplo.com` pelo e-mail real da equipe em `js/config.js` . - [] Reproduzir o defeito (Aula 41), corrigir, testar o caso e o fluxo ao redor, e fazer `git add .` e `git commit -m "..."` com mensagem clara. - [] Enviar a branch: `git push -u origin correcao-nome-do-defeito` . - [] No GitHub, clicar em **Compare & pull request** (em português: **Comparar e criar pull request**), escrever título e descrição (o que mudou, qual caso CT-xx corrige e como testar) e escolher outra integrante em **Reviewers** (em português: **Revisores**); clicar em **Create pull request** (em português: **Criar pull request**). - [] A revisora abre **Files changed** (em português: **Arquivos alterados**), lê o que mudou e clica em **Review changes** (em português: **Revisar alterações**) > **Approve** (em português: **Aprovar**) > **Submit review** (em português: **Enviar revisão**). - [] Quem abriu o pull request clica em **Merge pull request** (em português: **Mesclar pull request**), **Confirm merge** (em português: **Confirmar merge**) e **Delete branch** (em português: **Excluir branch**); depois roda `git switch main` e `git pull` . - [] Em dupla, fazer a revisão de código com a lista de conferência da aula (nomes em português sem acento, comentários que explicam o porquê, um arquivo por responsabilidade, nenhum `innerHTML` com dados, nenhum segredo no código, mensagens em português, "Carregando..." e `try/catch` em toda chamada ao banco, valores em R\$ e datas `dd/mm/aaaa` , código repetido em um lugar só). - [] (Opcional, em dupla) Criar um conflito de propósito no `README.md` em duas branches e resolver com **Resolve conflicts** (em português: **Resolver conflitos**), **Mark as resolved** (em português: **Marcar como resolvido**) e **Commit merge** (em português: **Fazer o commit do merge**). - [] Atualizar o `docs/TESTES.md` com os casos que passaram a `OK` . - [] Rodar `git log --oneline --graph` e conferir as branches e os merges. - **Etiqueta sugerida:** Testes, Configuração · Essencial - **Depende de:** D14-A41 - **Aula e requisitos:** docs/guia-aulas/dia14-aula42-correcoes-e-git-em-equipe.md · RF-21 e os RF dos casos que falharam

Dia 15 · 27/10/2026 · terça-feira

Marco: Marco 4 · MVP publicado, versão 0.9 (tag `v0.9`). Evidência: link publicado e avaliação prática dos 5 indicadores da UC3.

Entrega do dia: sistema publicado no GitHub Pages, README completo com diagrama ER e MVP revisado e demonstrado ao vivo.

Observações do dia: - A Aula 45 tem quatro blocos de 15 minutos (checklist, demonstração, conferência do que falta e avaliação dos indicadores) mais a tag. Se o tempo estourar, o **ensaio da demonstração** (Passo 2) é o que fica mais curto: cronometrar para caber em 8 minutos e, se

preciso, repetir antes da próxima aula. Os itens de segurança da checklist (nenhum `innerHTML` com dados, nenhuma chave `service_role`, RLS ligada) não são pulados. - Os requisitos que ainda **não existem** no fim do dia são RF-11, RF-20, RF-22 e RF-25 (entram no Dia 26): a equipe cria um cartão de pendência no quadro para cada item "falta" ou "parcial" da tabela da Aula 45, com prioridade. - A cada `git push` na `main`, o GitHub Pages publica de novo em alguns minutos; evitar enviar alterações no meio da demonstração. - Se um indicador ficar "parcialmente atendido" ou "não atendido", a recuperação é imediata nas aulas seguintes; anotar o que falta no quadro. - As aulas dos Dias 16 a 25 tratam de outros assuntos e não fazem parte deste backlog: depois do Dia 15, o próximo cartão é o D26-A76, em 12/11/2026.

Data: 27/10/2026 - terça-feira - **Cartão (Título no Trello):** D15-A43 – Publicar o site no GitHub Pages e liberar o endereço publicado no Supabase - **User Story:** Como cliente, Quero abrir o site pelo celular em um link público, Para que eu faça meus pedidos de qualquer lugar, inclusive recuperando a senha pelo e-mail. - **Critérios de Aceite:** - O endereço `https://SEU-USUARIO.github.io/vitrine-col/` abre a página inicial com estilo, imagens, carrossel e dados do Supabase. - O fluxo de pedido funciona online: catálogo, sacola, login, finalizar e botões de WhatsApp (CT-07). - O link do e-mail de recuperação de senha abre a tela da senha nova no **site publicado** (RF-24 online). - No celular, as telas funcionam sem rolagem horizontal. - Procurar `service_role` não encontra nenhuma chave; `href="/` e `src="/` dão 0 resultados; `innerHTML` em `js/` dá 0 resultados. - **Checklist interna do cartão:** - [] Procurar `service_role` e `secret` com Ctrl+Shift+F (em português: **Localizar nos arquivos**; Mac: Cmd+Shift+F): só o comentário do `js/config.js`, e no `config.js` só a URL e a chave `anon`. - [] Procurar `href="/` e `src="/`: 0 resultados (usar caminhos como `css/base.css`). - [] Procurar `innerHTML` em `js/`: 0 resultados. - [] Conferir que a `main` está atualizada (`git status` diz `nothing to commit` e `git pull` não traz nada). - [] No GitHub, abrir **Settings** (em português: **Configurações**) > **Pages** (em português: **Páginas**); em **Build and deployment** (em português: **Construção e implantação**) > **Source** (em português: **Origem**), escolher **Deploy from a branch** (em português: **Implantar a partir de uma branch**). - [] Em **Branch**, escolher `main` e a pasta `/ (root)` (em português: `/ (raiz)`) e clicar em **Save** (em português: **Salvar**). - [] Esperar de 1 a 3 minutos e copiar o endereço que aparece em **Your site is live at** (em português: **Seu site está no ar em**). - [] No Supabase, abrir **Authentication** (em português: **Autenticação**) > **URL Configuration** (em português: **Configuração de URL**) e trocar **Site URL** (em português: **URL do site**) pelo endereço publicado, com a barra no fim. - [] Em **Redirect URLs** (em português: **URLs de redirecionamento**), acrescentar (sem apagar os de teste) `https://SEU-USUARIO.github.io/vitrine-col/recuperar-senha.html` e clicar em **Save** (em português: **Salvar**). - [] No computador, percorrer catálogo, produto, sacola, **Entrar**, finalizar um pedido (CT-07) e o painel da lojista no endereço publicado. - [] Testar "Esqueci minha senha" online com o e-mail de um membro da organização e conferir que o link abre o site publicado. - []

No celular, abrir o mesmo endereço e testar o menu, o carrossel (arrastar o dedo), a escolha de tamanho e a sacola. - [] Se algo não carregar, abrir F12 > **Console** e a aba **Network** (em português: **Rede**) e olhar os erros 404 (atenção a maiúsculas e minúsculas nos nomes de pastas e arquivos). - [] Colar no `README.md` uma linha `Site: https://...`. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Publicação, Segurança · Essencial - **Depende de:** D14-A42 - **Aula e requisitos:** docs/guia-aulas/dia15-aula43-publicacao-no-github-pages.md · RF-24 (online)

Data: 27/10/2026 - terça-feira - **Cartão (Título no Trello):** D15-A44 – Escrever o README completo com diagrama ER e instruções de uso - **User Story:** Como aluna desenvolvedora, Quero um README com tecnologias, passo a passo, telas e diagrama das 8 tabelas, Para que qualquer pessoa nova consiga rodar o projeto só lendo o repositório. - **Critérios de Aceite:** - O README no GitHub mostra todas as seções, com tabelas e o **diagrama ER** desenhado (bloco `mermaid`). - O endereço do site publicado e os nomes das integrantes estão corretos (`SEU-USUARIO` foi trocado). - Uma colega que não participou da escrita seguiu "Como rodar" e "Como configurar o Supabase" só com o README, em outra pasta, e chegou ao sistema funcionando. - A lista de telas bate com os arquivos reais do projeto (as telas de conta, de pedidos da cliente e de pedidos recebidos só existem depois do Dia 26 e o README avisa). - O diagrama tem as 8 tabelas e as ligações iguais às do `01_schema.sql`. - **Checklist interna do cartão:** - [] Substituir **todo** o conteúdo do `README.md` pelo texto da aula (o que é, tecnologias, como rodar, como configurar o Supabase, telas, diagrama ER e instruções para cliente e lojista). - [] Trocar `SEU-USUARIO` pelo usuário do GitHub e escrever o nome de cada integrante. - [] Conferir que o bloco do diagrama abre com três crases seguidas de `mermaid` e fecha com três crases. - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - [] Abrir o repositório no GitHub e conferir o título, as listas, as tabelas e o diagrama (a pré-visualização do VS Code, **Ctrl+Shift+V**; Mac: Cmd+Shift+V, não desenha o diagrama). - [] Pedir a uma colega que não participou da escrita para seguir o README em outra pasta (clone em outro lugar) e anotar onde ela travou. - [] Conferir a tabela de telas com os arquivos reais da raiz do projeto. - [] Conferir o diagrama com o `01_schema.sql` (8 tabelas e ligações). - [] Corrigir o README com o que a colega apontou e, se precisar, usar **Preview** (em português: **Visualização**) na tela de edição do GitHub. - [] Rodar `git add .`, `git commit -m "..."` e `git push` com as correções. - **Etiqueta sugerida:** Documentação · Importante - **Depende de:** D15-A43 - **Aula e requisitos:** docs/guia-aulas/dia15-aula44-documentacao-tecnica.md · não se aplica (documentação do projeto)

Data: 27/10/2026 - terça-feira - **Cartão (Título no Trello):** D15-A45 – Revisar o MVP, demonstrar o fluxo completo e marcar a versão 0.9 (Marco 4) - **User Story:** Como aluna desenvolvedora, Quero revisar o MVP com uma checklist, demonstrar o caminho do catálogo ao pedido e provar cada

indicador com evidências, Para que a versão 0.9 fique publicada e as pendências sigam para o Dia 26. - **Critérios de Aceite:** - A checklist técnica está toda marcada (ou os itens pendentes viraram cartões no quadro): 0 `innerHTML` com dados, nenhuma chave `service_role`, mensagens em português e nada em branco no modo **Offline**, 360 px e 1280 px sem rolagem horizontal, `label` em todo campo e `alt` em toda imagem, valores em R\$ e datas `dd/mm/aaaa`, RLS ligada nas 8 tabelas e política `dev: envio de fotos` inexistente, README completo, todas as integrantes com commits (`git shortlog -sn`). - A demonstração completa (vitrine, filtros, produto, sacola de duas lojas, login, finalização, prova no Supabase de dois pedidos com o mesmo `grupo_id` e status `novo`, painel da lojista e bloqueio da cliente no painel) foi feita em até 8 minutos, sem erros. - A tabela de requisitos foi preenchida ("funciona", "parcial" ou "falta") e cada "falta" ou "parcial" virou cartão no quadro com prioridade (RF-11, RF-20, RF-22 e RF-25 ficam para o Dia 26). - Cada um dos 5 indicadores tem evidência apresentada e menção registrada. - A tag `v0.9` aparece no GitHub (**Marco 4:** MVP publicado). - **Checklist interna do cartão:** - [] Abrir em abas diferentes o link público, o repositório, o Supabase e o quadro (são as evidências). - [] Em dupla, conferir cada item da checklist técnica no sistema publicado e no repositório (busca com Ctrl+Shift+F; Mac: Cmd+Shift+F). - [] Preparar os dados da demonstração antes: duas lojas, uma conta de cliente e produtos com fotos. - [] Ensaiar o roteiro da aula, com uma integrante conduzindo e outra narrando: (1) visitante na página inicial, **Vestidos**, filtro de tamanho e busca; (2) produto, troca de foto pelo teclado, sacola agrupada por loja; (3) **Finalizar sacola**, login, volta à sacola e **Pedidos enviados**; (4) prova no Supabase (dois pedidos, mesmo `grupo_id`, status `novo`); (5) lojista: **Minha loja**, **Meus produtos**, cadastrar produto com 3 fotos, editar e desativar; (6) segurança: cliente bloqueada no painel e preço forjado recusado pelo console. - [] Cronometrar a demonstração para caber em 8 minutos. - [] Comparar o sistema com a lista de requisitos e escrever "funciona", "parcial" ou "falta" para RF-01 a RF-05, RF-06 e RF-07, RF-08, RF-09, RF-10, RF-12 a RF-14, RF-15 a RF-19, RF-21, RF-23 e RF-24. - [] Criar um cartão no quadro, com prioridade, para cada requisito "falta" ou "parcial" (RF-11, RF-20, RF-22 e RF-25 entram no Dia 26). - [] Apresentar a evidência de cada um dos 5 indicadores (ambiente de desenvolvimento; melhores práticas da linguagem; elaboração de código; compilação e depuração; integração com banco de dados) e registrar a menção. - [] Se algum indicador ficar parcial ou não atendido, anotar o que falta no quadro. - [] Com a `main` atualizada, rodar `git switch main`, `git pull`, `git tag v0.9` e `git push origin v0.9` (só uma integrante cria a tag). - [] Conferir a tag `v0.9` na lista de tags do GitHub. - **Etiqueta sugerida:** Testes, Publicação · Essencial - **Depende de:** D15-A44 - **Aula e requisitos:** docs/guia-aulas/dia15-aula45-revisao-geral-e-avaliacao-da-uc3.md · revisão de RF-01 a RF-10, RF-12 a RF-19, RF-21, RF-23 e RF-24 · **Marco 4**

Dia 26 · 12/11/2026 · quinta-feira

Entrega do dia: sistema com conteúdo real e acompanhamento do pedido: a lojista atualiza o status e deixa recado, a cliente é avisada dentro do sistema, a conta pode ser excluída e o catálogo tem lojas, produtos e fotos reais.

Observações do dia: - Este dia começa a fase de fechamento da versão 1.0. Antes de abrir o primeiro cartão, conferir que o MVP 0.9 continua publicado, que existe **pelo menos um pedido** gravado na tabela `pedidos` e que a lojista dona dele tem a loja cadastrada. - **O que a aula pede além dos 60 minutos da Aula 78 (fica só como observação, sem cartão extra):** a carga do conteúdo real pelo próprio sistema (meta: pelo menos 3 lojas e 12 produtos, com 2 fotos cada; e 13 ou mais produtos ativos no total para o CT-22), a limpeza dos dados de teste e a revisão de responsividade e acessibilidade com o conteúdo real em 360 e 1280 px, com um cartão de pendência para cada problema. Dividir entre as integrantes: duas pessoas ficam com **Minha conta** (Aula 78) e as outras com a carga de conteúdo. Só usar fotos e textos com autorização de uso e nunca imagens com rosto de pessoas sem consentimento. - **Itens pendentes do backlog (RF-11, RF-20, RF-22, RF-25 e os cartões criados no D15-A45):** priorizar os **Essenciais** e os **Importantes**; os **Desejáveis** só entram se sobrar tempo. - **Se o tempo apertar, o primeiro a sair é o aviso da lojista à cliente por WhatsApp** (o aviso dentro do sistema continua; é o RF-22 principal). O botão "Avisar a cliente pelo WhatsApp" só aparece quando a cliente tem telefone cadastrado. - Cada uma das três aulas termina com uma branch própria, pull request e revisão de outra integrante (como no D14-A42); o commit direto na `main` não vale a partir de hoje. - Os testes de acesso indevido (CT-18 e CT-20) e o CT-17 são feitos hoje e voltam a ser refeitos no Dia 28.

Data: 12/11/2026 - quinta-feira - **Cartão (Título no Trello):** D26-A76 – Criar a tela Pedidos recebidos e o fluxo de status da lojista - **User Story:** Como lojista, Quero ver os pedidos da minha loja e mudar o status deixando um recado, Para que a cliente saiba se o pedido foi confirmado, concluído ou cancelado. - **Critérios de Aceite:** - A lojista vê **só** os pedidos da própria loja, cada um com código `#xxxxxxx`, data, selo de status, nome da cliente, itens, **Total do pedido**, campo "Recado para a cliente (opcional)" e botões; outra lojista não vê esses pedidos e, sem pedidos, aparece "Sua loja ainda não recebeu nenhum pedido." (RF-20, RN-09). - Em pedido **novo** aparecem só **Confirmar reserva**, **Cancelar pedido** e **Salvar só o recado**; em **confirmado**, **Marcar como concluído**, **Cancelar pedido** e **Salvar só o recado**; em concluído ou cancelado, só **Salvar só o recado** (RN-05). - Status e recado são salvos juntos, a mensagem "Pedido #...: reserva confirmada. A cliente será avisada em Meus pedidos." aparece e o banco deixa o pedido com `status_visto = false` (RN-13). - Pelo Console, alterar o total é recusado com "Só o status e o recado do pedido podem ser alterados." (CT-18); pular de **novo** direto para **concluído** é recusado com "Esta mudança de status não é permitida para este pedido." e mexer em `status_visto` é recusado (CT-20). - Datas aparecem em dd/mm/aaaa. - **Checklist interna do cartão:** - [] Abrir um novo branch

para a aula: `git switch -c pedidos-recebidos` . - [] Conferir que existe pelo menos um pedido na tabela `pedidos` e que a lojista dona dele tem a loja cadastrada. - [] Em `js/modelos/Pedido.js` , fazer as seis trocas na ordem da aula: rótulos antes de `export class Pedido {` ; campos novos depois de `#itens` ; construtor completo; leitores novos (`mensagemLoja` , `statusVisto` , `criadoEm` , `clienteNome` , `clienteTelefone`) antes de `grupoId` ; métodos `deLinha` , `statusPermitidos` , `proximosStatus` e `alterarStatus` antes de `codigoCurto` ; fim da classe a partir de `rotuloParaCliente()` . - [] Em `js/ui/formatadores.js` , colar `formatadorDeData` antes de `formatarPreco` e `formatarData` antes de `normalizarTelefone` . - [] No final de `js/servicos/pedidoServico.js` , colar o bloco de comentário e as constantes, `listarPedidosDaMinhaLoja` e `atualizarPedido` (muda status e recado numa única atualização). - [] Criar `painel-pedidos.html` e `js/paginas/painelPedidos.js` . - [] No `css/paginas.css` , colar a seção do botão de aviso antes de `/* ----- Botão "Carregar mais" do catálogo ----- */` . - [] Entrar como a **lojista dona** de um pedido, abrir **Painel > Pedidos recebidos** e conferir o cartão de cada pedido. - [] Em um pedido novo, escrever o recado `Separado, pode retirar amanhã` , clicar em **Confirmar reserva** e conferir o selo **Confirmado** e o **Recado atual para a cliente**. - [] Clicar em **Marcar como concluído** e conferir que só resta **Salvar só o recado**. - [] Em outro pedido novo, clicar em **Cancelar pedido**, confirmar a pergunta "Cancelar este pedido? A cliente será avisada em Meus pedidos." e conferir o selo **Cancelado**. - [] Conferir o botão **Avisar a cliente pelo WhatsApp** (só aparece se a cliente tem telefone); este botão é o primeiro a sair se faltar tempo. - [] Entrar como **outra lojista** e conferir que ela não vê esses pedidos (RN-09). - [] No **SQL Editor** (em português: **Editor SQL**), rodar `select status, mensagem_loja, status_visto, atualizado_em from public.pedidos;` e conferir `status_visto = false` nos pedidos mexidos. - [] Testar CT-18 no Console (F12 > **Console**): trocar `ID-DO-PEDIDO` por um pedido da sua loja, rodar o roteiro do `TESTES.md` e conferir a mensagem de recusa; repetir com a outra lojista e conferir `data: []` . - [] Testar CT-20 no Console com um pedido **novo**: pular etapa e mexer em `status_visto` . - [] Rodar `git add .` , `git commit -m "..."` e `git push -u origin pedidos-recebidos` ; abrir o pull request e pedir a revisão de outra integrante; depois do merge, `git switch main` e `git pull` . - **Etiqueta sugerida:** Fluxo de Pedido, Segurança · Importante - **Depende de:** D15-A45 - **Aula e requisitos:** docs/guia-aulas/dia26-aula76-pedidos-recebidos-e-fluxo-de-status.md · RF-20, RN-05, RN-09, RN-13; CT-18 e CT-20

Data: 12/11/2026 - quinta-feira - **Cartão (Título no Trello):** D26-A77 – Criar Meus pedidos e o aviso de novidades para a cliente - **User Story:** Como cliente, Quero ver os meus pedidos e ser avisada quando a lojista mudar o status ou deixar um recado, Para que eu saiba o que está acontecendo sem precisar perguntar pelo WhatsApp. - **Critérios de Aceite:** - Como cliente, o link **Meus pedidos** não tem número quando não há novidade; depois de a lojista confirmar um pedido, o link mostra um número (por exemplo, `1`) em até 60 segundos ou ao recarregar (RF-22, RN-13). -

Meus pedidos mostra as compras agrupadas ("Compra de 26/10/2026", **Total da compra**, "2 pedidos, um por loja"), do mais recente ao mais antigo, com o status em linguagem simples (por exemplo, **Reserva confirmada** e **Enviado à loja**) e o recado ("Recado da loja: Separado, pode retirar amanhã") (RF-11). - O pedido mexido pela lojista tem borda destacada e o selo **Atualizado**; logo depois que a tela abre, o número do cabeçalho some e, ao recarregar, o selo não aparece mais; `select status_visto from public.pedidos where status = 'confirmado';` volta `true` (CT-17). - Cliente sem pedidos vê "Você ainda não fez nenhum pedido. Ver o catálogo."; lojista que abre `meus-pedidos.html` volta para o painel; visitante vai para `login.html?voltar=meus-pedidos.html` e, depois de entrar como cliente, volta para **Meus pedidos**. - **Checklist interna do cartão:** - [] Abrir uma nova branch: `git switch -c meus-pedidos`. - [] No `pedidoServico.js`, colar `listarMeusPedidos` antes de `listarPedidosDaMinhaLoja` e `contarNovidades` e `marcarComoVistos` no final do arquivo. - [] No `cabecalho.js`, fazer as trocas da aula: começo do arquivo (importações e intervalo de consulta); `linksDaConta` com o contador no link **Meus pedidos**; colar `atualizarContadorDeNovidades`, `pararDeVerNovidades` e `comecarAVerNovidades` antes de `atualizarContadorSacola`; trocar `criarLink` e `montarCabecalho`. - [] Criar `meus-pedidos.html` e `js/paginas/meusPedidos.js`. - [] Abrir **dois navegadores** (ou uma janela anônima): um com a **lojista** e outro com a **cliente** dona de um pedido novo. - [] Como cliente, abrir qualquer página e conferir que **Meus pedidos** não tem número. - [] Como lojista, em **Pedidos recebidos**, escrever o recado e clicar em **Confirmar reserva**. - [] Como cliente, esperar até 60 segundos (ou recarregar) e conferir o número no link **Meus pedidos**. - [] Abrir **Meus pedidos** e conferir os grupos de compra, o selo **Atualizado**, o status e o recado; conferir que o número do cabeçalho some logo depois e que o selo não volta ao recarregar. - [] No **SQL Editor** (em português: **Editor SQL**), conferir `status_visto = true`. - [] Testar uma cliente sem pedidos, a lojista abrindo `meus-pedidos.html` e o visitante com volta ao fim do login. - [] Se o contador nunca aparecer, conferir que a lojista mudou o status ou salvou um recado depois da criação do pedido. - [] Se aparecer `permission denied for function marcar_pedidos_como_vistos`, rodar até o fim a Parte 2 do `01_schema.sql` (Aula 26). - [] Rodar `git add .`, `git commit -m "..."` e `git push -u origin meus-pedidos`; abrir o pull request e pedir a revisão. - **Etiqueta sugerida:** Fluxo de Pedido, JavaScript · Importante - **Depende de:** D26-A76 - **Aula e requisitos:** docs/guia-aulas/dia26-aula77-meus-pedidos-e-aviso-de-novidades.md · RF-11, RF-22, RN-13; CT-17

Data: 12/11/2026 - quinta-feira - **Cartão (Título no Trello):** D26-A78 – Criar Minha conta com exclusão da própria conta (EXCLUIR) - **User Story:** Como cliente, Quero excluir a minha conta e os meus dados digitando EXCLUIR, Para que eu possa apagar tudo de forma definitiva quando quiser. - **Critérios de Aceite:** - **Minha conta** mostra **Meus dados** (nome, e-mail e tipo de conta) e a área **Excluir minha conta** (borda vermelha) com a lista do que vai sumir (para a lojista, também a loja, os produtos, as fotos e os pedidos recebidos). - Com o campo vazio, aparece "Digite EXCLUIR para

confirmar." e nada é excluído; digitando `excluir` ou `EXCLUIR` (sem espaços), aparecem "Excluindo a conta..." e "Conta excluída. Obrigada por ter usado a VitrineCol. Redirecionando...", e em 2 segundos a pessoa volta à página inicial como visitante (RF-25). - A exclusão apaga perfil, pedidos, loja, produtos, tamanhos e fotos, inclusive os arquivos no Storage: a conta some de **Authentication > Users**, `select count(*) from public.perfis where nome = 'NOME DA CONTA';` volta `0` e a pasta da loja some do bucket `produtos` (RN-14, CT-24). - Tentar entrar com o e-mail e a senha da conta excluída mostra "E-mail ou senha incorretos."; visitante que abre `minha-conta.html` vai para `login.html?voltar=minha-conta.html`. - **Checklist interna do cartão:** - [] Abrir uma nova branch: `git switch -c minha-conta`. - [] No `authServico.js`, trocar a importação de formatadores pela versão da aula (com o apoio ao Storage) e colar no final a seção "Excluir a própria conta" (apaga primeiro os arquivos de foto do Storage e depois chama a função `excluir_minha_conta` do banco). - [] Criar `minha-conta.html` e `js/paginas/minhaConta.js`. - [] No `css/paginas.css`, colar a seção "Minha conta" antes de `/* ----- Botão "Carregar mais" do catálogo ----- */`. - [] Criar uma conta **só para o teste**: cadastrar uma lojista de teste em **Cadastrar**, criar a loja em **Minha loja** e um produto com **uma foto**. - [] Clicar no nome no cabeçalho ("Olá, ...") para abrir **Minha conta** e conferir os dados e a área de exclusão. - [] Clicar em **Excluir minha conta** com o campo vazio e conferir a mensagem; depois digitar `EXCLUIR` e confirmar. - [] Conferir em **Authentication > Users** (em português: **Autenticação > Usuários**), no **SQL Editor** (em português: **Editor SQL**) (`perfis` e `lojas`) e em **Storage > produtos** (em português: **Armazenamento**) que tudo sumiu. - [] Tentar entrar com a conta excluída e conferir a mensagem de erro. - [] Abrir `minha-conta.html` sem login e conferir o redirecionamento. - [] Se aparecer `permission denied for function excluir_minha_conta`, rodar o `04_melhorias.sql` (Aula 39); se as fotos ficarem no Storage, apagar a pasta da loja pelo painel. - [] Conferir que o pedido de exclusão chama a função do banco e que o site nunca grava direto nas tabelas `pedidos` e `itens_pedido`. - [] Rodar `git add .`, `git commit -m "..."` e `git push -u origin minha-conta`; abrir o pull request (o conteúdo real fica no Supabase, não no repositório) e pedir a revisão de outra integrante. - [] Combinar com a equipe a carga do conteúdo real e os ajustes de interface descritos nas observações do dia, registrando cada problema em um cartão de pendência com prioridade. - **Etiqueta sugerida:** Segurança, Supabase (Banco de Dados) · Importante - **Depende de:** D26-A77 - **Aula e requisitos:** docs/guia-aulas/dia26-aula78-minha-conta-conteudo-real-e-ajustes.md · RF-25, RN-14, RN-12 (carga de fotos); CT-24

Dia 27 · 13/11/2026 · sexta-feira

Entrega do dia: relatório de testes (`docs/relatorio-de-testes.md`) com os 25 casos executados no site publicado, as anotações de pelo menos 5 testes com usuárias externas e a lista priorizada de correções com responsável e prazo.

Observações do dia: - **O que a aula pede além dos 60 minutos (fica só como observação, sem cartão extra):** executar os 25 casos de teste em 25 minutos exige dividir os casos entre as duplas (por exemplo, CT-01 a CT-08, CT-09 a CT-16 e CT-17 a CT-25) e deixar os dados de teste prontos antes da aula; os casos que dependem de e-mail (CT-08 e CT-23) podem ser repetidos mais tarde se aparecer "Muitas tentativas em pouco tempo". - Na Aula 79 **não pode sobrar nenhum** **N/A** : o que depende de dados que faltam (13 ou mais produtos ativos, duas lojas, conta de teste para o CT-24) é preparado primeiro. Para o CT-24, usar sempre uma conta criada só para o teste, nunca uma conta real. - A Aula 80 depende de **pelo menos 5 usuárias externas** (de outras turmas, convidadas ou lojistas parceiras) disponíveis na hora; se não houver tantas, cada sessão leva só cerca de 10 minutos e a equipe convida colegas de outras turmas. Anotar só o primeiro nome da usuária e nunca dados pessoais. - Se o site cair durante uma sessão de teste, anotar o horário e a mensagem de erro: é um defeito **crítico**. - Na Aula 81, se a lista de correções ficar grande demais, **só os críticos e os importantes** entram nas Aulas 82 e 83; o restante vai para a seção "não será feito na versão 1.0" com o motivo. - A confirmação de e-mail do Supabase continua **desligada** durante os testes se o e-mail de confirmação não chegar para quem não é da equipe; registrar essa decisão e o risco no relatório (seção 5 da Aula 81).

Data: 13/11/2026 - sexta-feira - **Cartão (Título no Trello):** D27-A79 – Executar os 25 casos de teste no site publicado e escrever o roteiro de usabilidade - **User Story:** Como aluna desenvolvedora, Quero executar todos os casos de teste (CT-01 a CT-25) no site publicado e ter um roteiro de usabilidade pronto, Para que a equipe saiba o que funciona de verdade antes de chamar usuárias externas. - **Critérios de Aceite:** - Os dados de teste estão preparados no site publicado: duas lojas de duas lojistas diferentes (cada uma com WhatsApp e pelo menos 2 produtos ativos com fotos, um deles com um tamanho de estoque 0), 13 ou mais produtos ativos, uma conta de cliente, uma de lojista com pedido recebido e uma conta só para o teste de exclusão. - **Todos** os 25 casos do **docs/TESTES.md** têm **OK** ou **FALHOU** (nenhum **N/A**) e cada **FALHOU** tem uma observação; o quadro do topo (versão testada, endereço, data, aparelho, quem testou) está preenchido e a tabela **Resumo** está contada. - A decisão sobre **Confirm email** (em português: **Confirmar e-mail**) e o limite de e-mails por hora foram conferidos no Supabase e registrados. - O roteiro de usabilidade (tarefa lida em voz alta, tabela de observação e 3 perguntas finais) está pronto; a meta é que uma cliente nova vá do catálogo ao pedido em até 6 ações por loja. - **Checklist interna do cartão:** - [] Conferir no site **publicado** os dados de teste: duas lojas, 13 ou mais produtos ativos, cliente (de preferência com telefone), lojista com pelo menos um pedido recebido e uma conta só para o teste de exclusão (de preferência lojista com loja, produto e foto). - [] Conferir em **Authentication** (em português: **Autenticação**) > **URL Configuration** (em português: **Configuração de URL**) que o endereço de **recuperar-senha.html** do site publicado está liberado (CT-23). - [] Abrir o **docs/TESTES.md**, o site publicado e o Supabase em abas separadas e dividir os casos entre as

duplas. - [] Preencher o quadro do topo do **TESTES.md** (versão testada, endereço, data, navegador e aparelho, quem testou). - [] Executar CT-01 a CT-10 e escrever **OK** ou **FALHOU** (com observação) na coluna **Resultado**. - [] Executar CT-11 a CT-16, usando os roteiros do console do **TESTES.md** para CT-11 (F12 > **Console**, trocando só os ids em MAIÚSCULAS). - [] Executar CT-17 (lojista confirma o pedido com recado; cliente vê o contador, **Atualizado**, o status e o recado) e CT-18 a CT-21 com os roteiros do console. - [] Executar CT-22 (12 produtos, botão **Carregar mais produtos** sem repetir), CT-23 (recuperação de senha), CT-24 (exclusão da conta de teste) e CT-25 (foto grande com cerca de 200 KB no Storage). - [] Se sobrar algum **N/A**, preparar os dados que faltam e executar o caso de novo. - [] Contar **OK**, **FALHOU** e **N/A** e preencher a tabela **Resumo**. - [] No Supabase, abrir **Authentication** (em português: **Autenticação**) > **Providers** (em português: **Provedores**) > **Email** (em português: **E-mail**) e olhar **Confirm email** (em português: **Confirmar e-mail**); se for ligar, testar antes com um e-mail que não seja da equipe e, se o e-mail não chegar, deixar desligada e registrar o risco. - [] Procurar a página de limites (**Rate Limits**, em português: **Limites de taxa**) dentro de **Authentication** e anotar quantos e-mails por hora o projeto aceita. - [] Escrever o roteiro de usabilidade em um arquivo ou folha da equipe: a tarefa ("Você quer comprar um vestido e uma camiseta de duas lojas diferentes. Escolha as peças, junte tudo e faça o pedido. Fale em voz alta o que está pensando."), a tabela de observação (usuária, aparelho, tempo, ações, onde hesitou, erros, se concluiu sozinha, comentário) e as 3 perguntas finais (o que foi mais fácil, o que foi confuso, o que mudaria). - [] Rodar **git add .**, **git commit -m "..."** e **git push**. - **Etiqueta sugerida:** Testes, Documentação · Essencial - **Depende de:** D26-A78 - **Aula e requisitos:** docs/guia-aulas/dia27-aula79-plano-de-testes-e-roteiro-de-usabilidade.md · CT-01 a CT-25 (verificação de RF-01 a RF-25)

Data: 13/11/2026 - sexta-feira - **Cartão (Título no Trello):** D27-A80 – Aplicar o roteiro de usabilidade com pelo menos 5 usuárias externas e registrar cada problema - **User Story:** Como aluna desenvolvedora, Quero observar pessoas de fora usando o sistema sem ajudar, Para que a equipe saiba em qual tela e em qual passo elas travam. - **Critérios de Aceite:** - Pelo menos **5 usuárias externas** (que não são da equipe) fizeram a tarefa, com a tabela de observação de cada sessão preenchida: tempo, ações, onde hesitou, erros, se concluiu sozinha e comentários (RF-10, RF-11, RF-20, RF-22 no fluxo completo). - Cada problema está registrado em uma tabela única da equipe (Nº, tela, passo, o que aconteceu, primeiro nome da usuária e frase dela), com tela e passo precisos. - Os números foram calculados: quantas usuárias testaram, quantas concluíram sozinhas, tempo médio do catálogo ao botão de WhatsApp e média de ações (meta: até 6 por loja). - A tabela de observação está guardada em **docs/** (sem nomes completos nem dados pessoais). - **Checklist interna do cartão:** - [] Combinar os papéis de cada sessão: condutora (lê a tarefa e faz as perguntas finais), observadora (anota em silêncio) e cronometrista; combinar o sinal de silêncio (observadora de braços cruzados). - [] Preparar de 3 a 5 estações (computador ou celular) com o

site publicado aberto na página inicial, sacola vazia e uma conta de cliente de teste (ou sem conta, para testar também o cadastro). - [] Dar a cada observadora uma cópia da tabela de observação e das 3 perguntas finais. - [] Acolher a usuária e ler o consentimento ("Vamos testar o sistema, não você... Eu não vou poder ajudar... Pode ser?"); só começar depois do "sim". - [] Ler a tarefa em voz alta, sem explicar, e disparar o cronômetro quando a usuária começar; parar quando ela chegar ao botão do WhatsApp (ou desistir). - [] Observadora: contar cada clique ou escolha como uma ação, anotar hesitações de mais de 5 segundos, cliques errados, mensagens de erro e frases entre aspas. - [] Se a usuária travar por mais de 2 minutos, perguntar só "O que você está procurando?" e anotar que houve ajuda. - [] Fazer as 3 perguntas finais e anotar as respostas. - [] Logo depois de cada sessão, passar a limpo cada problema em uma linha da tabela única (tela e passo precisos) e registrar também as sessões que correram bem. - [] Esvaziar a sacola e voltar à página inicial entre as sessões. - [] Se houver lojista parceira ou colega, aplicar a segunda tarefa: cadastrar um produto com uma foto e confirmar um pedido com recado, medindo o tempo. - [] Somar os resultados (quantas testaram, quantas concluíram sozinhas, tempo médio e média de ações) e comparar com a meta. - [] Guardar a tabela na pasta `docs/` e rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Testes · Importante - **Depende de:** D27-A79 - **Aula e requisitos:** docs/guia-aulas/dia27-aula80-testes-com-usuarias-reais.md · RF-10, RF-11, RF-20, RF-22 (fluxo completo)

Data: 13/11/2026 - sexta-feira - **Cartão (Título no Trello):** D27-A81 – Consolidar o relatório de testes e priorizar as correções em crítico, importante e desejável - **User Story:** Como aluna desenvolvedora, Quero juntar os problemas em uma tabela, classificar a gravidade e definir quem corrige o quê, Para que a equipe corrija primeiro o que mais atrapalha a cliente. - **Critérios de Aceite:** - O `docs/relatorio-de-testes.md` tem as 5 seções preenchidas: casos de teste, usabilidade, problemas, lista priorizada e o que não entra na versão 1.0. - A seção 1 mostra os 25 casos contados (sem `N/A`) e cada falha explicada com o requisito e a decisão. - A seção 3 tem cada problema com tela, passo, número de usuárias e gravidade (crítico, importante ou desejável); problemas repetidos estão agrupados em uma linha. - A seção 4 tem a lista **ordenada** (primeiro os críticos), com um responsável e um prazo por item (no máximo 2 itens por integrante na Aula 82), e cada item é um cartão no quadro com o título `[crítico] tela: problema`. - A seção 5 registra o que não será feito na 1.0, com o motivo, e a decisão sobre a confirmação de e-mail e o limite de e-mails. - **Checklist interna do cartão:** - [] Criar `docs/relatorio-de-testes.md` com o modelo da aula e preencher o cabeçalho (versão, endereço, período e equipe). - [] Contar no `docs/TESTES.md` os `OK`, `FALHOU` e `N/A` e preencher a tabela da seção 1; se sobrar `N/A`, executar o caso agora. - [] Para cada caso `FALHOU`, preencher o que falhou, o requisito (RF ou RN em palavras) e a decisão. - [] Com as tabelas de observação da Aula 80, calcular quantas usuárias testaram, quantas concluíram sozinhas, o tempo médio e a média de ações, e comparar com a meta de até 6 ações

por loja (seção 2). - [] Juntar em uma tabela única os casos que falharam e os problemas das usuárias, agrupando os repetidos (por exemplo, "3 usuárias") (seção 3). - [] Classificar a gravidade com as perguntas da aula, nesta ordem: impede sacola, finalização ou WhatsApp, ou expõe dados? (crítico); a usuária conseguiu, mas demorou ou errou? (importante); é estética ou texto? (desejável). Se houver discordância, vale a gravidade mais alta. - [] Ordenar a lista priorizada (críticos primeiro, os que mais usuárias tiveram antes) e escrever um responsável e um prazo por item: críticos na Aula 82, importantes na Aula 83 ou no tempo que sobrar (seção 4). - [] Criar um cartão no quadro para cada item, com o título `[crítico] tela: problema` e, na descrição, tela, passo, caso de teste a repetir e quem corrige. - [] Cortar a lista para só críticos e importantes nas Aulas 82 e 83 e mover o resto para a seção 5 com o motivo (por exemplo, "pagamento on-line: fora do escopo do curso"). - [] Registrar na seção 5 a decisão sobre a confirmação de e-mail e o limite de e-mails (Aula 79). - [] Rodar `git add .`, `git commit -m "..."` e `git push`. - **Etiqueta sugerida:** Testes, Documentação · Essencial - **Depende de:** D27-A80 - **Aula e requisitos:** docs/guia-aulas/dia27-aula81-feedbacks-e-priorizacao-das-correcoes.md · CT-01 a CT-25 (e os requisitos afetados pelos problemas encontrados)

Dia 28 · 16/11/2026 · segunda-feira

Marco: Marco 8 · versão 1.0 publicada e congelada (tag `v1.0` e release no GitHub). Evidência: link final, documentação e relatório de testes.

Entrega do dia: problemas críticos corrigidos e retestados, segurança revisada, versão final publicada, documentação final e código congelado (a partir de agora, só correção de erro grave).

Observações do dia: - **O que a aula pede além dos 60 minutos (fica só como observação, sem cartão extra):** na Aula 82, a revisão das mensagens das **15 telas** em todos os estados da tabela da aula (o que é dividido entre as duplas) e a varredura de código por `PROVISÓRIO`, `alert(`, `console.log(` e `throw new Error(`; na Aula 83, o teste extra do visitante pelo Console; na Aula 84, a proteção opcional da `main` (branch protection) e o vídeo curto da demonstração (gravação de reserva). - Todas as correções entram por **branch, pull request revisado por outra integrante e merge**. A tag `v1.0` só é criada **depois** do merge do README final (a tag deve apontar para o commit que já tem o README final); se for criada antes, apagar (`git tag -d v1.0` e `git push origin --delete v1.0`) e criar de novo. - A publicação no GitHub Pages leva alguns minutos depois do merge; refazer os casos de teste com **Ctrl+Shift+R** (Mac: Cmd+Shift+R) para ignorar o cache. - Se aparecer um problema que não é crítico, ele **não** entra hoje: vai para a seção "não será feito na versão 1.0" do relatório de testes. - Depois de hoje, o código fica **congelado**: só correção de erro

grave, em branch e pull request revisado. - As aulas seguintes (Dias 29 a 31) fazem parte de outro material e não têm cartão neste backlog.

Data: 16/11/2026 - segunda-feira - **Cartão (Título no Trello):** D28-A82 – Corrigir os problemas críticos em branches e refazer os casos de teste - **User Story:** Como aluna desenvolvedora, Quero corrigir cada problema crítico em uma branch e provar que ele acabou refazendo o caso de teste, Para que a versão final não tenha falhas que impeçam o pedido ou exponham dados. - **Critérios de Aceite:** - Cada problema crítico da lista tem uma branch, um pull request **aprovado por outra integrante** e o merge na `main`; o cartão está em "Concluído". - Cada caso de teste correspondente foi **refeito no site publicado** com resultado `OK` e o `docs/TESTES.md` foi atualizado (`FALHOU` para `OK` , com data). - A tabela das 15 telas foi percorrida: nenhuma tela fica em branco e nenhuma mostra mensagem em inglês (RF-21). - A varredura de código em `js/` não encontrou `PROVISÓRIO` , `alert(` nem `throw new Error(` (deve ser `ErroApp` com mensagem em português); só `console.warn` e `console.error` do registro de erros são permitidos. - **Checklist interna do cartão:** - [] Rer o cartão do problema: tela, passo, o que aconteceu e o caso de teste a repetir. - [] Reproduzir o problema seguindo o cartão e anotar a mensagem do Console (F12 > **Console**) e do pedido que falhou (aba **Network**, em português: **Rede**). - [] Voltar à `main` (`git switch main`), trazer as novidades (`git pull`) e criar a branch da correção (`git switch -c correcao-...`). - [] Achar a causa com o Console e com **Ctrl+Shift+F** (em português: **Localizar nos arquivos**; Mac: Cmd+Shift+F). Se o pedido na aba **Network** tem erro, o problema é do banco ou da regra; se deu certo, é da tela. - [] Corrigir o mínimo; se a correção é em regra de negócio (preço, estoque, status), fazer na classe ou no serviço, não na página. - [] Repetir exatamente os passos do caso (CT-xx) e depois os casos vizinhos (por exemplo, se mexeu na sacola, refazer CT-04, CT-05 e CT-07). - [] Fazer `git add .` , `git commit -m "..."` (mensagem clara) e `git push -u origin correcao-...` . - [] Abrir o pull request (**Compare & pull request**, em português: **Comparar e criar pull request**), citar o caso de teste e o cartão e pedir a revisão de outra integrante. - [] Ao revisar o pull request de outra integrante, abrir **Files changed** (em português: **Arquivos alterados**) e conferir: mudança mínima; nomes em português sem acento e comentários que explicam o porquê; nenhum `innerHTML` com dados, nenhum segredo e nenhum `console.log` de teste; mensagens em português; regra em um só lugar; passos de teste feitos. Aprovar (**Approve**, em português: **Aprovar**) ou comentar. - [] Depois do merge, atualizar a `main` (`git switch main` e `git pull`), esperar a publicação e refazer o caso no site publicado; atualizar o `docs/TESTES.md` e mover o cartão para "Concluído". - [] Dividir as 15 telas entre as duplas e provocar em cada uma os estados da tabela da aula (por exemplo, **Offline** na aba **Rede**, `loja.html?id=abc` , `produto.html?id=abc` , sacola vazia, senha errada, link de recuperação já usado, `minha-conta.html` sem login); abrir um cartão para cada falha. - [] Varrer `js/` por `PROVISÓRIO` (não pode sobrar nada), `alert(` , `console.log(` e `throw new Error(` . - [] Se a

correção ficar grande, dividir em dois pull requests ou passar a parte menos grave para "não será feito na 1.0". - **Etiqueta sugerida:** Testes, JavaScript · Essencial - **Depende de:** D27·A81 - **Aula e requisitos:** docs/guia-aulas/dia28-aula82-correcao-dos-problemas-criticos.md · RF-21 e os RF dos casos que falharam

Data: 16/11/2026 - segunda-feira - **Cartão (Título no Trello):** D28·A83 – Revisar a segurança (RLS, chaves, acessos indevidos) e publicar a versão final - **User Story:** Como cliente, Quero que ninguém consiga alterar meus pedidos nem os produtos de outra loja, Para que meus dados e minhas compras fiquem protegidos na versão final. - **Critérios de Aceite:** - As consultas de conferência deram: 8 tabelas com `rowsecurity = true` ; 24 regras (categorias 1, itens_pedido 1, lojas 4, pedidos 2, perfis 3, produto_fotos 4, produtos 5, tamanhos 4); política provisória de fotos ausente (consulta vazia); 0 regras de INSERT direto em pedidos e itens; 3 funções; limites do bucket de `2097152` bytes e 3 tipos de imagem. - A busca por `service_role` e `secret` não encontra nenhuma chave (só o comentário do `js/config.js`), `innerHTML` em `js/` dá 0 resultados, `PROVISÓRIO` e `alert(` dão 0, e o histórico (`git log -S"service_role" --oneline`) não mostra chave de verdade. - Pelo console, o banco **recusa** tudo: a lojista B não altera o produto da lojista A (`data: []` e erro de RLS no Storage, CT-11); a lojista não altera o total do pedido (CT-18); não pula etapa de status nem mexe no aviso da cliente (CT-20); a cliente não grava pedido direto nem com preço forjado (CT-21); o visitante não lê pedidos nem perfis (`data: []`). - O site publicado está na versão final e o link do e-mail de recuperação de senha abre o **site publicado** (CT-23), com **Site URL** e **Redirect URLs** do Supabase apontando para o endereço publicado. - **Checklist interna do cartão:** - [] No **SQL Editor** (em português: **Editor SQL**), rodar as consultas 1.1 (RLS nas 8 tabelas), 1.2 (regras por tabela), 1.3 (política provisória de fotos), 1.4 (INSERT direto em pedidos e itens) e 1.5 (funções e limites do bucket) e comparar com o esperado. - [] Se algum resultado for diferente, **parar:** é crítico. Rodar de novo `database/02_rls.sql` e depois `database/04_melhorias.sql` e repetir as consultas. - [] Procurar `service_role` e `secret` (só o comentário do `config.js`), `innerHTML` em `js/` (0), `PROVISÓRIO` e `alert(` (0). - [] No terminal, rodar `git log -S"service_role" --oneline` ; se aparecer um commit que adicionou uma chave de verdade, gerar uma chave nova em **Project Settings** (em português: **Configurações do projeto**) > **API** e apagar a antiga. - [] Conferir que o arquivo `.env` (se existir) está no `.gitignore` e não foi para o GitHub. - [] Preparar duas lojistas (A e B), cada uma com loja, produto e um pedido recebido, e uma cliente; trocar os textos em MAIÚSCULAS pelos ids reais. - [] Refazer CT-11 como lojista B no Console (F12 > **Console**) e conferir `data: []` nas quatro primeiras linhas e o erro de RLS na foto; conferir que nada mudou no painel da lojista A. - [] Refazer CT-18 como lojista dona e como outra lojista (`data: []`). - [] Refazer CT-20 com um pedido novo da sua loja: pular etapa e mexer em `status_visto` . - [] Refazer CT-21 como **cliente** e conferir com `select count(*) from public.pedidos;` que o total de pedidos não aumentou. - [] Fazer o teste extra como visitante

(sem entrar): as duas consultas devolvem `data: []`. - [] Conferir que todos os pull requests estão na `main` e que o GitHub Pages terminou de publicar (em **Settings > Pages**, em português: **Configurações > Páginas**, ou na aba **Actions**, em português: **Ações**); recarregar com **Ctrl+Shift+R** (Mac: Cmd+Shift+R). - [] No Supabase, abrir **Authentication** (em português: **Autenticação**) > **URL Configuration** (em português: **Configuração de URL**): **Site URL** (em português: **URL do site**) igual ao endereço publicado e **Redirect URLs** (em português: **URLs de redirecionamento**) com `https://SEU-USUARIO.github.io/vitrine-col/recuperar-senha.html` (os endereços de teste podem sair). - [] Fazer o fluxo de recuperação de senha (CT-23) no site publicado com uma conta de teste, conferindo que o link do e-mail abre o site publicado. - [] Percorrer no site publicado: página inicial, catálogo, sacola, login, pedido, **Meus pedidos** e o painel da lojista. - [] Atualizar o `docs/TESTES.md` (resultados novos de CT-11, CT-18, CT-20, CT-21 e CT-23, com data), criar a branch `revisao-de-seguranca`, commitar e abrir o pull request. - **Etiqueta sugerida:** Segurança, Publicação · **Essencial** - **Depende de:** D28-A82 - **Aula e requisitos:** docs/guia-aulas/dia28-aula83-revisao-de-seguranca-e-publicacao.md · RN-05, RN-06, RN-09, RN-13; CT-11, CT-18, CT-20, CT-21

Data: 16/11/2026 - segunda-feira - **Cartão (Título no Trello):** D28-A84 – Fechar a documentação, marcar a versão 1.0 e congelar o código (Marco 8) - **User Story:** Como aluna desenvolvedora, Quero entregar a versão 1.0 marcada no Git, com documentação que bate com o sistema e o código congelado, Para que qualquer pessoa consiga usar e entender o projeto sem a nossa ajuda. - **Critérios de Aceite:** - O `README.md` está atualizado para a versão 1.0, com manual de uso (cliente e lojista), estrutura de pastas, segurança, testes, histórico de versões e o diagrama ER das 8 tabelas desenhado no GitHub. - Uma colega que não escreveu o manual o executou como cliente (do catálogo a **Meus pedidos**) e como lojista (loja, produto, pedido recebido) e tudo funcionou exatamente como escrito. - A tag `v1.0` existe no GitHub e aponta para o commit que **já tem** o README final; há um release com as novidades da versão. - O combinado de congelamento ("a partir de hoje, só correção de erro grave, em branch e pull request revisado") está registrado no README e no quadro. - O link público funciona no computador e no celular, no fluxo completo, com o e-mail de contato real na página de privacidade (**Marco 8**). - **Checklist interna do cartão:** - [] Criar a branch `documentacao-final` (`git switch -c documentacao-final`). - [] Substituir **todo** o conteúdo do `README.md` pela versão da aula; trocar `SEU-USUARIO`, conferir os nomes das integrantes e a data da versão e ler cada passo conferindo se é verdadeiro. - [] Pedir a uma colega que não participou da escrita que execute o manual como cliente e como lojista; anotar o que não bate e corrigir. - [] Conferir que os 15 arquivos HTML da tabela de telas existem na raiz. - [] Comparar a árvore de pastas do README com a pasta real (`js/modelos`, `js/servicos`, `js/ui`, `js/paginas`, `css` com 5 arquivos e `database` com 4 scripts). - [] Fazer `git add .`, `git commit -m "..."`, `git push -u origin documentacao-final`, abrir o pull request, pedir a revisão e fazer o

merge **antes** de marcar a tag. - [] Conferir no GitHub que o diagrama ER aparece desenhado (bloco com três crases e `mermaid` bem fechado). - [] Registrar o combinado de congelamento no README (seção "Histórico de versões") e no quadro. - [] Com a `main` atualizada (`git switch main` e `git pull`), criar e enviar a tag (`git tag v1.0` e `git push origin v1.0`); só uma integrante cria a tag. - [] No GitHub, abrir **Releases** (em português: **Versões**) > **Create a new release** (em português: **Criar uma nova versão**); em **Choose a tag** (em português: **Escolher uma tag**) escolher `v1.0` ; em **Release title** (em português: **Título da versão**) escrever `Versão 1.0` ; descrever as novidades e clicar em **Publish release** (em português: **Publicar versão**). - [] (Opcional) Em **Settings** (em português: **Configurações**) > **Branches**, criar uma regra para a `main` que exija pull request antes de juntar. - [] Abrir <https://SEU-USUARIO.github.io/vitrine-col/> no computador e no celular, em janela anônima ou depois de sair da conta. - [] Percorrer o fluxo completo: página inicial, catálogo com filtros, produto, sacola com duas lojas, **Cadastrar**, finalizar, **Meus pedidos** e, em outro aparelho, o painel da lojista com **Pedidos recebidos**. - [] Conferir que os links do README, do release e do cabeçalho funcionam e que o e-mail de contato da página de privacidade é o real. - [] Guardar o link e um vídeo curto da demonstração (gravação de reserva, para o caso de a internet falhar). - **Etiqueta sugerida:** Documentação, Publicação · Essencial - **Depende de:** D28-A83 - **Aula e requisitos:** docs/guia-aulas/dia28-aula84-documentacao-final-e-versao-1-0.md · não se aplica (documentação e entrega da versão 1.0) · **Marco 8**

Anexo: cobertura dos requisitos funcionais (RF-01 a RF-25)

Todos os 25 requisitos têm pelo menos um cartão. Nenhum ficou fora do escopo deste backlog.

Requisito	O que o sistema faz	Prioridade	Cartões que o entregam
RF-01	Listar os produtos ativos em cards, 12 por página, com "Carregar mais produtos"	Essencial	D7-A19, D10-A29, D10-A30 (visual: D4-A12)
RF-02	Filtrar o catálogo por tipo de roupa	Essencial	D7-A20, D10-A30
RF-03	Filtrar o catálogo por loja	Essencial	D7-A20, D10-A30
RF-04	Filtrar por tamanho, só com estoque nesse tamanho	Importante	D7-A20, D10-A30
RF-05	Buscar produtos pelo nome, sem diferenciar maiúsculas de minúsculas	Desejável	D7-A20, D10-A30
RF-06	Página da loja com nome, descrição, endereço, WhatsApp e produtos	Essencial	D12-A34 (estrutura: D3-A9)
RF-07	Botão com o link do mapa da loja	Desejável	D12-A34
RF-08	Página do produto com galeria, descrição, preço, loja, tamanhos e quantidade	Essencial	D12-A34
RF-09	Sacola com peças de várias lojas, subtotal por loja e total	Essencial	D12-A35 (estrutura: D5-A15)
RF-10	Finalizar a sacola, um pedido por loja e um botão de WhatsApp por loja	Essencial	D12-A36 (parte), D13-A39
RF-11	Mostrar à cliente os pedidos dela, agrupados por compra	Importante	D26-A77
RF-12	Cadastrar usuária com nome, e-mail, senha, perfil e telefone opcional	Essencial	D13-A37 (estrutura: D5-A15)

Requisito	O que o sistema faz	Prioridade	Cartões que o entregam
RF-13	Entrar e sair da conta, mantendo a sessão	Essencial	D13·A37 (estrutura: D3·A8, D3·A9)
RF-14	Controlar o acesso às telas conforme o perfil	Essencial	D13·A39
RF-15	A lojista cria e edita a própria loja	Essencial	D11·A31
RF-16	A lojista cadastra produto com tamanhos e estoque	Essencial	D11·A31
RF-17	A lojista edita produto	Essencial	D11·A32
RF-18	A lojista desativa ou exclui produto	Essencial	D11·A32, D11·A33
RF-19	Enviar até 5 fotos por produto, escolher a capa e reordenar	Importante	D11·A33
RF-20	Mostrar à lojista os pedidos da loja, mudar o status e deixar um recado	Importante	D26·A76
RF-21	Avisar carregamento, erro e sucesso, sem deixar tela em branco	Essencial	D6·A18, D7·A19, D7·A21, D8·A24, D10·A28 a D10·A30, D14·A40, D14·A42, D28·A82
RF-22	Avisar a cliente, dentro do sistema, quando a lojista muda o status ou deixa recado	Importante	D26·A77
RF-23	Página inicial de vitrine com hero, carrossel, tipos de roupa, destaques e lojas	Importante	D14·A40
RF-24	Recuperar a senha por e-mail	Importante	D13·A38, D15·A43
RF-25	A usuária exclui a própria conta	Importante	D26·A78